

Chapter 08

2026 주요정보통신기반시설 기술적 취약점 분석·평가 방법 상세가이드

DBMS

1. 계정 관리	596
2. 접근 관리	628
3. 옵션 관리	646
4. 패치 관리	663

01 DBMS 서버 취약점 분석 · 평가 항목

1. 계정 관리

점검항목	항목 중요도	항목코드
기본 계정의 비밀번호, 정책 등을 변경하여 사용	상	D-01
데이터베이스의 불필요 계정을 제거하거나, 잠금설정 후 사용	상	D-02
비밀번호의 사용기간 및 복잡도를 기관의 정책에 맞도록 설정	상	D-03
데이터베이스 관리자 권한을 꼭 필요한 계정 및 그룹에 대해서만 허용	상	D-04
비밀번호 재사용에 대한 제약 설정	중	D-05
DB 사용자 계정을 개별적으로 부여하여 사용	중	D-06
root 권한으로 서비스 구동 제한	중	D-07
안전한 암호화 알고리즘 사용	상	D-08
일정 횟수의 로그인 실패 시 이에 대한 잠금정책 설정	중	D-09

2. 접근 관리

점검항목	항목 중요도	항목코드
원격에서 DB 서버로의 접속 제한	상	D-10
DBA 이외의 인가되지 않은 사용자가 시스템 테이블에 접근할 수 없도록 설정	상	D-11
안전한 리스너 비밀번호 설정 및 사용	상	D-12
불필요한 ODBC/OLE-DB 데이터 소스와 드라이버를 제거하여 사용	중	D-13
데이터베이스의 주요 설정파일, 비밀번호 파일 등과 같은 주요 파일들의 접근 권한이 적절하게 설정	중	D-14
관리자 이외의 사용자가 오라클 리스너의 접속을 통해 리스너 로그 및 trace 파일에 대한 변경 제한	하	D-15
Windows 인증 모드 사용	하	D-16

3. 옵션 관리

점검항목	항목 중요도	항목코드
Audit Table은 데이터베이스 관리자 계정에 접근하도록 제한	하	D-17
응용프로그램 또는 DBA 계정의 Role이 Public으로 설정되지 않도록 조정	상	D-18
OS_ROLES, REMOTE_OS_AUTHENTICATION, REMOTE_OS_ROLES를 FALSE로 설정	상	D-19
인가되지 않은 Object owner의 제한	하	D-20
인가되지 않은 GRANT OPTION 사용 제한	중	D-21
데이터베이스의 자원 제한 기능을 TRUE로 설정	하	D-22
xp_cmdshell 사용 제한	상	D-23
Registry Procedure 권한 제한	상	D-24

4. 패치 관리

점검항목	항목 중요도	항목코드
주기적 보안 패치 및 벤더 권고 사항 적용	상	D-25
데이터베이스의 접근, 변경, 삭제 등의 감사 기록이 기관의 감사 기록 정책에 적합하도록 설정	상	D-26

D-01 (상)	DBMS > 1. 계정 관리
	기본 계정의 비밀번호, 정책 등을 변경하여 사용
개요	
점검 내용	DBMS 기본 계정의 초기 비밀번호 및 권한 정책을 변경하여 사용하는지 점검
점검 목적	DBMS 기본 계정의 초기 비밀번호 및 권한 정책 변경 사용 유무를 점검하여 비인가자의 초기 비밀번호 대입 공격을 차단하고 있는지 확인하기 위함
보안 위협	DBMS 기본 계정 초기 비밀번호 및 권한 정책을 변경하지 않을 경우 비인가자가 인터넷 통해 DBMS 기본 계정의 초기 비밀번호를 획득하여 초기 비밀번호를 그대로 사용하고 있는 DB에 접근하여 기본 계정에 부여된 권한의 취약점을 이용하여 DB 정보를 유출할 수 있는 위험이 존재함
참고	※ 기본 계정 : DB 설치 후 초기에 기본으로 생성되어있는 DBMS 관리용 계정(예 : sa)
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, MySQL, Altibase, Tiber, PostgreSQL, Cubrid 등
판단 기준	양호 : 기본 계정의 초기 비밀번호를 변경하거나 잠금설정된 경우
	취약 : 기본 계정의 초기 비밀번호 를 변경하지 않거나 잠금설정을 하지 않은 경우
조치 방법	기본(관리자) 계정의 초기 비밀번호 및 권한 정책 변경
조치 시 영향	불필요한 기본 계정의 사용 제한

점검 및 조치 사례

● Oracle DB

[기본 계정 비밀번호 변경]

Step 1) 기본 계정 사용 여부 및 정책 확인

```
SQL> SELECT USERNAME, ACCOUNT_STATUS, PROFILE FROM DBA_USERS;
```

Step 2) 사용되는 계정의 경우 비밀번호 변경 후 사용

```
SQL> ALTER USER <기본 계정명> IDENTIFIED BY <신규 비밀번호>;
```

[기본 계정 잠금 설정]

Step 1) 활성화 되어 있는 기본 계정 확인

```
SQL> SELECT USERNAME, ACCOUNT_STATUS, PROFILE FROM DBA_USERS;
```

Step 2) 활성화 되어 있는 기본 계정 잠금 설정

```
SQL> SELECT username, account_status, lock_date, expiry_date, profile FROM dba_users WHERE  
account_status='OPEN';
```

```
ALTER USER <기본 계정명> ACCLUNT LOCK;
```

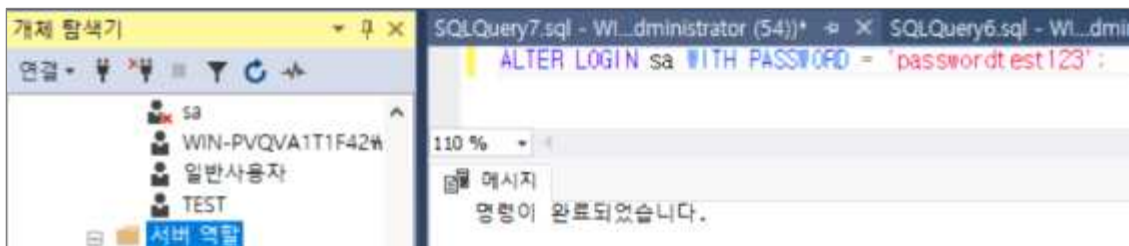
※ Oracle DB 기본 계정 정보

User	Password	User	Password
scott	tiger or tiger	system	manager
dbstmp	dbstmp	sys	changeon_install
tracesvr	trace	outln	outln
ordplugins	ordplugins	ordsys	ordsys
ctxsys	ctxsys	mdsys	mdsys
adams	wood	blake	paper
clark	clth	jones	steel
lbacsys	lbacsys	-	-

● MSSQL

Step 1) sa 계정 비밀번호 변경

ALTER LOGIN sa WITH PASSWORD = '신규 비밀번호';



[sa 계정 비밀번호 변경]

Step 2) 비밀번호 정책 강제 사용 적용

● MySQL

Step 1) root 계정 비밀번호 변경

[mysql 5.7]

```
mysql> UPDATE user SET authentication_string = PASSWORD('신규 비밀번호') WHERE User = 'root';
mysql> flush privileges;
```

[mysql 8.0]

```
mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY '신규 비밀번호';
```

```
mysql> flush privileges;
```

● Altibase

Step 1) 비밀번호 정책 설정 여부 확인

```
SELECT * FROM system_.sys_users_;
```

Step 2) ALTER USER 명령어로 비밀번호 변경

Altibase 서버에 sys 유저로 접속 후 ALTER USER 명령어로 비밀번호를 변경

```
ALTER USER sys IDENTIFIED BY [신규 비밀번호];
```

또는

altipasswd 명령어로 비밀번호 변경(Altibase 서버 온라인 상태에서 수행)

```
$ altipasswd
```

```
Previous Password : old_password
```

```
New Password : new_password
```

```
Retype New Password : new_password
```

● Tiberio

Step 1) sys 계정 비밀번호 변경

```
ALTER USER sys IDENTIFIED BY [신규 비밀번호];
```

● PostgreSQL

Step 1) postgres 계정으로 접속 계정 변경 및 접속

```
$ sudo -u postgres psql
```

```
# ALTER USER postgres WITH PASSWORD '신규 비밀번호';
```

```
# \q
```

● Cubrid

Step 1) 사용자 계정 비밀번호 사용 여부 확인

```
csql> SELECT name, password FROM db_user;
```

```
csql> SELECT * FROM db_password;
```

Step 2) 사용자 계정 비밀번호 변경

```
csql> ALTER USER "사용자 계정명" PASSWORD '신규 비밀번호';
```

※ 비밀번호가 취약하게 설정된 경우 비밀번호를 다음 기준을 준수하여 변경함

<비밀번호 관리 방법>

1. 영문, 숫자, 특수문자를 조합하여 계정명과 상이한 8자 이상의 비밀번호 설정
다음의 문자 종류 중 2가지 종류 이상을 조합하여 최소 10자리 이상 또는, 3가지 종류 이상을 조합하여 최소 8자리 이상의 길이로 구성
 - 1) 영문 대문자(26개)
 - 2) 영문 소문자(26개)
 - 3) 숫자(10개)
 - 4) 특수문자(32개)
2. 시스템마다 상이한 비밀번호 사용
3. 비밀번호를 기록해 놓을 경우 변형하여 기록
4. 가급적 자주 비밀번호를 변경

D-02 (상)	DBMS > 1. 계정 관리
	데이터베이스의 불필요 계정을 제거하거나, 잠금설정 후 사용
개요	
점검 내용	DBMS에 존재하는 계정 중 DB 관리나 운용에 사용하지 않는 불필요한 계정이 존재하는지 점검
점검 목적	불필요한 계정 존재 유무를 점검하여 불필요한 계정 정보(비밀번호)의 유출 시 발생할 수 있는 비인가자의 DB 접근에 대비되어 있는지 확인하기 위함
보안 위협	DB 관리나 운용에 사용하지 않는 불필요한 계정이 존재할 경우, 비인가자가 불필요한 계정을 이용하여 DB에 접근하여 데이터를 열람, 삭제, 수정할 위험이 존재함
참고	※ 불필요한 계정 : SCOTT, PM, ADAMS, CLARK 등의 Demonstration 계정 및 퇴사나 직무 변경 등으로 더 이상 사용하지 않는 계정
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, MySQL, Altibase, Tiberio, PostgreSQL, Cubrid 등
판단 기준	양호 : 계정 정보를 확인하여 불필요한 계정이 없는 경우
	취약 : 인가되지 않은 계정, 퇴직자 계정, 테스트 계정 등 불필요한 계정이 존재하는 경우
조치 방법	계정별 용도를 파악한 후 불필요한 계정 삭제
조치 시 영향	Demonstration 계정 / Object 사용 불가 / 삭제된 계정 사용 불가

점검 및 조치 사례

● Oracle DB

Step 1) 불필요한 계정 및 Object 삭제

```
SQL> DROP USER [삭제할 계정];
```

● MSSQL

Step 1) 불필요한 계정 삭제

```
EXEC sp_droplogin '삭제할 계정';
```

● MySQL

Step 1) 불필요한 계정 삭제

```
DROP USER '삭제할 계정'@'호스트명 or IP';
FLUSH PRIVILEGES;
```


● Altibase

Step 1) 모든 사용자 확인

```
SELECT * FROM system_.sys_users_;
```

Step 2) 불필요한 계정 삭제

```
DROP USER user_name CASCADE;
```

● Tiberio

Step 1) 모든 사용자 확인

Tiberio에서는 사용자의 정보를 제공하기 위해 아래 나열된 정적 뷰를 제공하고 있으며, DBA나 일반 사용자 모두 사용할 수 있다.

```
SELECT * FROM all_users;
```

```
SELECT * FROM dba_users;
```

```
SELECT * FROM user_users;
```

정적 뷰	설명
ALL_USERS	데이터베이스의 모든 사용자의 기본적인 정보를 조회하는 뷰
DBA_USERS	데이터베이스의 모든 사용자의 자세한 정보를 조회하는 뷰
USER_USERS	현재 사용자의 정보를 조회하는 뷰

Step 2) 불필요한 계정 삭제

```
DROP USER user_name CASCADE;
```

● PostgreSQL

Step 1) 모든 사용자 확인

쿼리문 조회 : `SELECT * FROM system_.sys_users_;`

명령어 조회 : `\du`

Step 2) 불필요한 계정 삭제

```
DROP ROLE '삭제할 계정';
```

- Cubrid

Step 1) 사용자 계정 목록 확인

```
csql> SELECT name, password FROM db_user;
```

Step 2) 불필요한 계정 삭제

```
csql> DROP USER [삭제할 계정];
```

D-03 (상)	DBMS > 1. 계정 관리
	비밀번호 사용 기간 및 복잡도를 기관의 정책에 맞도록 설정
개요	
점검 내용	기관 정책에 맞게 비밀번호 사용 기간 및 복잡도 설정이 적용되어 있는지 점검
점검 목적	비밀번호 사용 기간 및 복잡도 설정 유무를 점검하여 비인가자의 비밀번호 추측 공격(무차별 대입 공격, 사전 대입 공격 등)에 대한 대비가 되어있는지 확인하기 위함
보안 위협	비밀번호 사용 기간 및 복잡도 설정이 되어있지 않으면 비인가자가 비밀번호 추측 공격을 통해 획득한 계정의 비밀번호를 이용하여 DB에 접근할 수 있는 위험이 존재함
참고	※ 무차별 대입 공격(Brute Force Attack): 특정 암호를 해독하기 위해 가능한 모든 값을 대입하는 공격 방법 ※ 사전 대입 공격(Dictionary Attack): 사전에 있는 단어를 입력하여 비밀번호를 알아내거나 암호를 해독하는데 사용되는 컴퓨터 공격 방법
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, MySQL, Altibase, Tiber, PostgreSQL 등
판단 기준	양호 : 기관 정책에 맞게 비밀번호 사용 기간 및 복잡도 설정이 적용된 경우
	취약 : 기관 정책에 맞게 비밀번호 사용 기간 및 복잡도 설정이 적용되지 않은 경우
조치 방법	기관 정책에 맞게 비밀번호 사용 기간 및 복잡도 정책 설정
조치 시 영향	주기적인 비밀번호 변경 필요

점검 및 조치 사례

● Oracle DB

Step 1) PASSWORD_LIFE_TIME Profile 파라미터 변경

```
SQL> ALTER PROFILE <프로파일명> LIMIT PASSWORD_LIFE_TIME xx;
```

Step 2) Profile 값과 관련된 사용자 변경

```
SQL> ALTER PROFILE <계정명> PROFILE <변경할 프로파일명>;
```

Step 3) 비밀번호 정책 설정 변경

```
SQL> ALTER PROFILE <프로파일명> LIMIT
```

```
FAILED_LOGIN_ATTEMPTS 3 (비밀번호 실패 3번 까지만 가능)
```

```
PASSWORD_LIFE_TIME 30 (30일 동안만 비밀번호 사용 가능)
```

```
PASSWORD_REUSE_TIME 30 (사용한 비밀번호 30일 후부터 재사용 가능)
```

```
PASSWORD_VERIFY_FUNCTION verify_function (비밀번호 복잡성 검증)
```

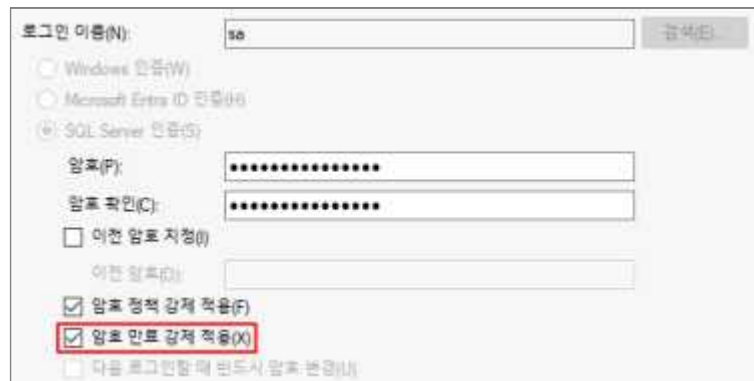
PASSWORD_GRACE_TIME 5; (life time이 끝나고 5일 동안 메시지를 보여줌)

● MSSQL

Step 1) 비밀번호 변경 주기는 '암호 만료 강제 적용'을 적용함으로써 주기적으로 변경할 수 있으며, 변경 기간은 OS의 '암호 정책'에서 적용받으므로 '암호 정책 > 최대 암호 사용 기간' 설정도 변경해야 함

Step 2) 암호 만료 강제 적용

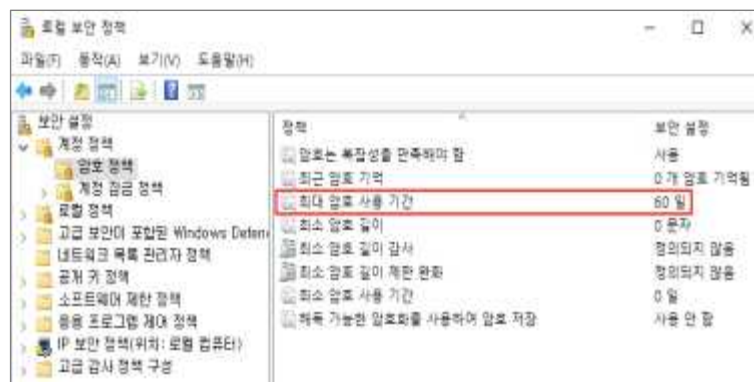
보안 > 로그인 > 각 로그인 계정 > 속성 > “암호 만료 강제 적용” 설정



[암호 만료 강제 적용 설정]

Step 3) OS 암호 정책 설정

[관리 도구] > [로컬 보안 정책] > [보안 설정] > [계정 정책] > [암호 정책] > 최대 암호 사용 기간 : '60일' 설정



[최대 암호 사용 기간 설정]

● MySQL

[비밀번호 복잡도 정책 설정]

Step 1) 비밀번호 정책 확인

```
mysql> SHOW VARIABLES LIKE 'validate_password%';
```

※ component_validate_password가 설치되어 있지 않은 경우 아래와 같이 해당 컴포넌트 설치

```
mysql> INSTALL COMPONENT 'file://component_validate_password';
```

Step 2) 비밀번호 정책 설정

다음과 같은 방법으로 각각의 비밀번호 정책을 설정

```
SET GLOBAL validate_password.policy = 'MEDIUM';
```

(비밀번호 정책의 강도 LOW/MEDIUM/STRONG)

```
SET GLOBAL validate_password.length = 8; (비밀번호 최소 길이)
```

```
SET GLOBAL validate_password.mixed_case_count = 1; (포함되어야 하는 영문 대소문자 최소 개수)
```

```
SET GLOBAL validate_password.number_count = 1; (포함되어야 하는 숫자 최소 개수)
```

```
SET GLOBAL validate_password.special_char_count = 1; (포함되어야 하는 특수문자 최소 개수)
```

※ Linux계열(/etc/my.cnf 또는 /etc/mysql/my.cnf), Windows(C:\ProgramData\MySQL\MySQL Server <설치된 버전>\my.ini)의 <mysqld> 섹션에 설정을 추가하여 정책 설정 가능

※ 비밀번호 신규 적용 및 초기화 시 설정 규칙에 맞추어 관리하고, 저장 시에는 일방향 암호화 알고리즘을 통해 암호화 처리(One-Way Encryption)함

[비밀번호 LifeTime 정책 적용]

Step 1) 비밀번호 정책 확인

```
mysql> SHOW VARIABLES LIKE 'default_password_lifetime';
```

Step 2) 비밀번호 LifeTime 설정

```
mysql> SET GLOBAL default_password_lifetime=90;
```

※ 기본 값

- 5.7.11 이전 버전 : 0

- 5.7.11 이후 버전 및 8.0 이후 버전 : 360

Step 3) 정책 적용전에 생성된 계정의 LifeTime 변경

```
mysql> ALTER USER <계정명>@<호스트명 or IP> PASSWORD EXPIRE INTERVAL 91 DAY;
```

● Altibase

Step 1) 다음 명령어를 통해 비밀번호 정책 설정 여부 확인

```
SELECT * FROM system_.sys_users_;
```

Step 2) 아래 Property에 대해 비밀번호 정책 설정

CASE_SENSITIVE_PASSWORD = 1
 FAILED_LOGIN_ATTEMPTS
 PASSWORD_LOCK_TIME
 PASSWORD_LIFE_TIME
 PASSWORD_GRACE_TIME
 PASSWORD_REUSE_TIME
 PASSWORD_REUSE_MAX
 PASSWORD_VERIFY_FUNCTION

정책 적용 시 다음 명령어를 사용

ALTER USER 계정명 LIMIT (Property 숫자);

예시) ALTER USER TESTUSER LIMIT (FAILED_LOGIN_ATTEMPTS 7, PASSWORD_LOCK_TIME 7);

● Tibero

Step 1) 사용자별 비밀번호 PROFILE 적용 여부 확인

비밀번호 설정 규칙에 맞추어 비밀번호를 설정할 수 있도록 시스템 차원에서 기능 제공

SELECT * FROM dba_users;

#	USERNAME	USER_ID	PASSWORD	ACCOUNT...	LOCK_DATE	EXPIRY_DATE	DEFAULT...	CREATED	PROFILE	DEFAULT...
1	SYSCAT	11	p45ud4F8...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP
2	SYSGIS	12	ZDn9smH5...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP
3	OUTLN	13	vNnIwIqd...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
4	TIBERO	18	+N2TQ1DX...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
5	TIBERO2	19	+N2TQ1DX...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
6	SYS	0	Ip7ny7YD...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP

[사용자별 비밀번호 PROFILE 적용 여부 확인]

Step 2) 설정되어 있을 경우 PROFILE 설정 내용 확인

SELECT * FROM dba_profiles;

#	PROFILE	RESOURCE_NAME	RESOURCE_TYPE	LIMIT
1	DEFAULT	FAILED_LOGIN_ATTEMPTS	PASSWORD	UNLIMITED
2	DEFAULT	PASSWORD_LIFE_TIME	PASSWORD	UNLIMITED
3	DEFAULT	PASSWORD_REUSE_TIME	PASSWORD	UNLIMITED
4	DEFAULT	PASSWORD_REUSE_MAX	PASSWORD	UNLIMITED
5	DEFAULT	PASSWORD_VERIFY_FUNCTION	PASSWORD	NULL_VERIFY...
6	DEFAULT	PASSWORD_LOCK_TIME	PASSWORD	1
7	DEFAULT	PASSWORD_GRACE_TIME	PASSWORD	UNLIMITED
8	DEFAULT	LOGIN_PERIOD	PASSWORD	UNLIMITED

[PROFILE 설정 내용 확인]

Step 3) 설정되어 있지 않을 경우 PROFILE 생성 또는 수정 시(ALTER PROFILE) 비밀번호 정책 설정 적용 시 다음 명령어를 사용

CREATE PROFILE prof LIMIT

예시) CREATE PROFILE prof LIMIT

failed_login_attempts 3

password_lock_time 1/1440

password_life_time 90

password_reuse_time unlimited

password_reuse_max 10

password_grace_time 10

password_verify_function verify_function;

D-04 (상)	DBMS > 1. 계정 관리
	데이터베이스 관리자 권한을 꼭 필요한 계정 및 그룹에 대해서만 허용
개요	
점검 내용	관리자 권한이 필요한 계정 및 그룹에만 관리자 권한을 부여하였는지 점검
점검 목적	관리자 권한이 필요한 계정과 그룹에만 관리자 권한을 부여하였는지 점검하여 관리자 권한의 남용을 방지하여 계정 유출로 인한 비인가자의 DB 접근 가능성을 최소화하고자 함
보안 위협	관리자 권한이 필요한 계정 및 그룹에만 관리자 권한을 부여하지 않으면 관리자 권한이 부여된 계정이 비인가자에게 유출될 경우 DB에 접근할 수 있는 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, MySQL, Altibase, Tiberio, PostgreSQL, Cubrid 등
판단 기준	양호 : 관리자 권한이 필요한 계정 및 그룹에만 관리자 권한이 부여된 경우
	취약 : 관리자 권한이 필요 없는 계정 및 그룹에 관리자 권한이 부여된 경우
조치 방법	관리자 권한이 필요한 계정 및 그룹에만 관리자 권한 부여
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) SYSDBA 권한 점검

```
SQL> SELECT username FROM v$pwfile_users WHERE username NOT IN (SELECT grantee FROM dba_role_privs WHERE granted_role='DBA') AND username != 'INTERNAL' AND SYSDBA = 'TRUE';
```

(어떠한 계정이라도 나오는 경우 취약)

Step 2) Admin에 부적합 계정 존재 여부 점검

```
SQL> SELECT grantee, privilege FROM dba_sys_privs WHERE grantee NOT IN ('SYS', 'SYSTEM', 'AQ_ADMINISTRATOR_ROLE', 'DBA', 'DSYS', 'BACSYS', 'HEDULER_ADMIN', 'MSYS') AND admin_option= 'YES' AND grantee NOT IN (SELECT grantee FROM dba_role_privs WHERE granted_role='DBA');
```

(어떠한 계정이라도 나오는 경우 취약)

Step 3) 관리자 권한이 불필요한 계정에서 관련 권한을 제거

```
SELECT * FROM DBA_SYS_PRIVS WHERE GRANTEE = '계정명';
```

불필요하게 시스템 권한을 부여한 계정의 권한 변경 필요

REVOKE <권한> FROM <계정명>;

시스템 권한 부여가 필요한 경우 필요한 테이블별 권한 부여

GRANT <권한> ON <테이블명> TO <계정명>;

인가된 사용자는 관리자 권한에 role을 grant한 후, 시스템 권한을 grant하고 role을 인가된 사용자에게 grant 함

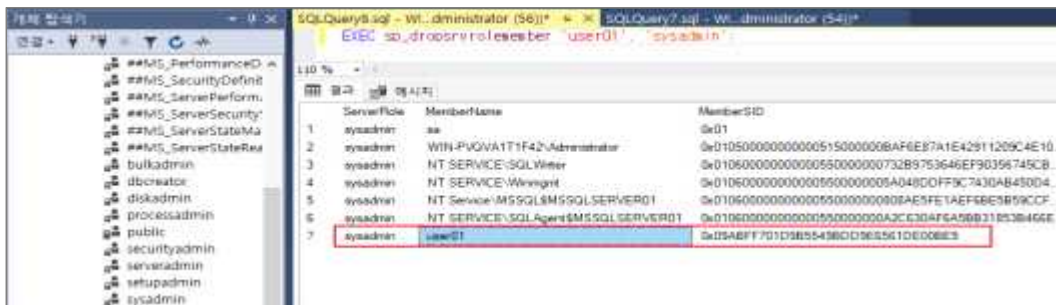
GRANT <Role_name> TO <계정명>;

● MSSQL

Step 1) sysadmin서버 역할의 계정 목록을 확인 후 서버 역할에 불필요한 계정이 있는 경우 서버 역할에서 삭제

EXEC sp_droprolemember 'user_name', 'sysadmin';

예시) EXEC sp_dropsvrolemember 'user01', 'sysadmin'; (user01계정을 sysadmin서버 역할에서 삭제)



[서버 역할에서 불필요 계정 삭제 예시]

● MySQL

Step 1)

Step 1) SUPER 권한(관리자 권한)이 부여되어 있는 계정 확인

```
SELECT GRANTEE FROM INFORMATION_SCHEMA.USER_PRIVILEGES WHERE
PRIVILEGE_TYPE = 'SUPER';
```

Step 2) 불필요하게 SUPER 권한이 부여되어 있는 계정에 대해 SUPER 권한 회수

REVOKE SUPER ON *.* FROM '<계정명>;

FLUSH PRIVILEGES;

※ 만약 관리자 'test'@'localhost' 계정이 바이너리 로그 정리 및 시스템 변수 수정을 위해 SUPER 권한을 필요로 하는 경우 아래와 같은 명령문으로 필요한 권한으로 제한.

GRANT BINLOG_ADMIN, SYSTEM_VARIABLES_ADMIN ON *.* TO 'test'@'localhost';

```
REVOKE SUPER ON *.* FROM 'test'@'localhost';
FLUSH PRIVILEGES;
```

● Altibase

Step 3) 계정별 부여된 시스템 권한 목록에서 grantee 확인

```
SELECT grantee_id FROM system_.sys_grant_system_;
```

Step 4) 계정별 부여된 시스템 권한 목록에서 user_id 확인

```
SELECT user_id, user_name FROM system_.sys_users_;
```

Step 5) 계정별 부여된 시스템 권한 목록에서 priv_id 확인

```
SELECT priv_id, priv_name FROM system_.sys_privileges_;
```

Step 6) 일반 사용자 계정 생성 시 시스템에 의해 부여되는 기본 권한 외 입력된 경우 해당 권한 삭제

시스템에 의해 자동으로 부여되는 권한	privileged_id
create session	215
create table	217
create sequence	210
create procedure	205
create view	229
create trigger	241
create synonym	245
create materialized view	252
create library	256

● Tiberio

Step 1) 계정별 부여된 시스템 권한 목록 확인 후, 아래 명령어 모두 입력

```
SELECT * FROM dba_users;
```

```
SELECT * FROM dba_sys_privs;
```

Step 2) dba_users 결과값에서 시스템 계정, 일반 계정 확인

SELECT * FROM dba_users;

SELECT * FROM dba_users; DBMS Output

#	USERNAME	USER_ID	PASSWORD	ACCOUNT_S...	LOCK_DATE	EXPIRY_DATE	DEFAULT_T...	CREATED	PROFILE	DEFAULT_T...	AUTHENTIC...
1	SYSCAT	11	p45664FGF...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP	PASSWORD
2	SYSGTS	12	20h9sh5R...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP	PASSWORD
3	OUTLN	13	vHte1gUj...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP	PASSWORD
4	TIBERO	18	+N27Q10h...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP	PASSWORD
5	TIBERO1	19	+N27Q10h...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP	PASSWORD
6	TEST1	21	gdyk21LQ...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP	PASSWORD
7	SYS	8	1p7hYVDQ...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP	PASSWORD

[시스템 계정, 일반 계정 확인]

Step 3) dba_sys_privs 결과값에서 일반 계정임에도 시스템 권한을 불필요하게 부여받고 있는지 확인

SELECT * FROM dba_sys_privs;

SELECT * FROM dba_sys_privs; DBMS Output

#	GRANTEE	PRIVILEGE	ADMIN_OPTION
235	DBA	DROP ANY INDEXTYPE	YES
236	DBA	EXECUTE ANY INDEXTYPE	YES
237	DBA	ADMINISTER DATABASE TRIGGER	YES
238	TEST1	ALTER SYSTEM	NO
239	TEST1	CREATE SESSION	NO
240	TEST1	ALTER SESSION	NO
241	TEST1	CREATE TABLESPACE	NO
242	TEST1	ALTER TABLESPACE	NO
243	TEST1	DROP TABLESPACE	NO
244	TEST1	CREATE USER	NO
245	TEST1	ALTER USER	NO
246	TEST1	DROP USER	NO

[일반 계정 권한 불필요하게 부여되어 있는지 확인]

Step 4) 일반 계정에 불필요한 시스템 권한이 부여된 경우 권한 삭제

● PostgreSQL

Step 1) 계정의 용도 파악 후 불필요한 계정은 삭제, 새로운 계정 생성 시 적절한 권한을 부여하여 생성
모든 사용자 확인

쿼리문 조회 : SELECT * FROM pg_user; or SELECT username, usesuper FROM pg_shadow;

명령어 조회 : \du

Step 2) 불필요하게 관리자 권한이 부여된 경우 권한 회수

ALTER ROLE <계정명> NOSUPERUSER;

ALTER ROLE <계정명> NOCREATEROLE;

ALTER ROLE <계정명> NOCREATEDB;

ALTER ROLE <계정명> NOREPLICATION;

ALTER ROLE <계정명> NOBYPASSRLS;

Step 3)

● Cubrid

Step 1) 계정의 용도 파악 후 불필요한 계정은 삭제, 새로운 계정 생성 시 적절한 권한을 부여하여 생성

Step 2) DBA 권한을 가진 사용자 계정 확인

```
SELECT a.name FROM db_user a, table(direct_groups) AS t(roles) WHERE roles.name = 'DBA';
```

DBA 권한을 가진 사용자 중 권한이 불필요한 계정의 권한 회수 or 계정 삭제

1. DBA가 가진 권한 한 번에 회수

```
REVOKE ALL PRIVILEGES ON test FROM 'GRANT_TEST';
```

2. DBA가 가진 권한을 명시적으로 회수

```
REVOKE SELECT ON test FROM 'GRANT_TEST';
```

```
REVOKE INSERT ON test FROM 'GRANT_TEST';
```

```
REVOKE UPDATE ON test FROM 'GRANT_TEST';
```

```
REVOKE DELETE ON test FROM 'GRANT_TEST';
```

```
REVOKE ALTER ON test FROM 'GRANT_TEST';
```

```
REVOKE DROP ON test FROM 'GRANT_TEST';
```

```
REVOKE EXECUTE ON test FROM 'GRANT_TEST';
```

```
REVOKE INDEX ON test FROM 'GRANT_TEST';
```

```
REVOKE REFERENCES ON test FROM 'GRANT_TEST';
```

3. DBA 권한을 가진 불필요 계정 삭제

```
DROP USER 'GRANT_TEST';
```

Step 3) 필요할 경우 적절한 권한을 부여하여 새로운 계정 생성

```
CREATE USER [계정명] PASSWORD '비밀번호' GROUPS [그룹명];
```

예시) CREATE USER test_db PASSWORD 'password' GROUPS DB_USER;

D-05 (중)	DBMS > 1. 계정 관리
	비밀번호 재사용에 대한 제약 설정
개요	
점검 내용	비밀번호 변경 시 이전 비밀번호를 재사용할 수 없도록 비밀번호 제약 설정이 되어있는지 점검
점검 목적	비밀번호 재사용 제약 설정 적용 여부를 점검하여 비밀번호 변경 시 이전 비밀번호 재사용을 제약하여 형식적인 비밀번호 변경을 원천적으로 차단하기 위함
보안 위협	비밀번호 재사용 제약 설정이 적용되어 있지 않을 경우 비밀번호 변경 전 사용했던 비밀번호를 재사용함으로써 비인가자의 계정 비밀번호 추측 공격에 대한 시간을 더 많이 허용하여 비밀번호 유출 위험이 증가함
참고	※ 비밀번호 제약 설정 : 비밀번호 변경 시 이전에 사용했던 비밀번호를 재사용할 수 없게 하는 설정으로써 이전 암호 재사용 가능 기간(PASSWORD_REUSE_TIME), 이전 암호 재사용 가능 횟수(PASSWORD_REUSE_MAX) 등이 있음
점검 대상 및 판단 기준	
대상	Oracle DB, Altibase, Tiberio 등
판단 기준	양호 : 비밀번호 재사용 제한 설정을 적용한 경우
	취약 : 비밀번호 재사용 제한 설정을 적용하지 않은 경우
조치 방법	PASSWORD_REUSE_TIME, PASSWORD_REUSE_MAX 파라미터 설정
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) SQL*Plus 설정 확인

-- Check for both reuse max and reuse time not set

```
SELECT profile FROM DBA_PROFILES WHERE (resource_name = 'PASSWORD_REUSE_MAX' AND limit IN ('UNLIMITED', 'NULL')) OR (profile IN (SELECT profile FROM DBA_PROFILES WHERE resource_name = 'PASSWORD_REUSE_TIME') AND limit IN ('UNLIMITED', 'NULL'));
```

-- Check for reuse max with value that is less than allowed minimum

```
SELECT profile FROM DBA_PROFILES WHERE resource_name = 'PASSWORD_REUSE_MAX' AND limit NOT IN ('UNLIMITED', 'NULL') AND REGEXP_LIKE(limit, '[0-9]+$') -- Only consider numeric values AND TO_NUMBER(limit) < 10;
```

-- Check for reuse time that is less than allowed minimum

```
SELECT profile FROM DBA_PROFILES WHERE resource_name = 'PASSWORD_REUSE_TIME' AND
limit NOT IN ('UNLIMITED', 'NULL') AND REGEXP_LIKE(limit, '[0-9]+$') -- Only consider numeric values AND TO_NUMBER(limit) < 365;
```

Step 2) PASSWORD_REUSE_TIME 및 PROFILE 파라미터 수정

```
SQL> ALTER PROFILE default LIMIT password_reuse_time 365 password_reuse_max 10;
```

```
SQL> ALTER PROFILE [프로파일명] LIMIT password_reuse_time DEFAULT password_reuse_max
default;
```

● Altibase

Step 1) 다음 명령어를 통해 비밀번호 정책 설정 여부 확인

```
SELECT * FROM system_.sys_users_;
```

Step 2) 아래 Property에 대해 비밀번호 정책 설정

```
CASE_SENSITIVE_PASSWORD
FAILED_LOGIN_ATTEMPTS
PASSWORD_LOCK_TIME
PASSWORD_LIFE_TIME
PASSWORD_GRACE_TIME
PASSWORD_REUSE_TIME
PASSWORD_REUSE_MAX
PASSWORD_VERIFY_FUNCTION
```

정책 적용 시 다음 명령어를 사용

```
ALTER USER [계정명] LIMIT (Property 숫자);
```

예시) ALTER USER TESTUSER LIMIT (FAILED_LOGIN_ATTEMPTS 7, PASSWORD_LOCK_TIME 7);

● Tiberio

Step 1) 사용자별 비밀번호 PROFILE 적용 여부 확인

SELECT * FROM dba_users;

#	USERNAME	USER_ID	PASSWORD	ACCOUNT_...	LOCK_DATE	EXPIRY_DATE	DEFAULT_...	CREATED	PROFILE	DEFAULT_...
1	SYSCAT	11	p45Ud4F6...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP
2	SYSG15	12	Z0n9sMHS...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP
3	OUTLN	13	vNhIeIqd...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
4	TIBERO	18	+N2TQ1DX...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
5	TIBERO1	19	+N2TQ1DX...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
6	SYS	0	lp7nY7YD...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP

[사용자별 비밀번호 PROFILE 적용 여부 확인]

Step 2) 설정되어 있을 경우 PROFILE 설정 내용 확인

SELECT * FROM dba_users;

#	PROFILE	RESOURCE_NAME	RESOURCE_...	LIMIT
1	DEFAULT	FAILED_LOGIN_ATTEMPTS	PASSWORD	UNLIMITED
2	DEFAULT	PASSWORD_LIFE_TIME	PASSWORD	UNLIMITED
3	DEFAULT	PASSWORD_REUSE_TIME	PASSWORD	UNLIMITED
4	DEFAULT	PASSWORD_REUSE_MAX	PASSWORD	UNLIMITED
5	DEFAULT	PASSWORD_VERIFY_FUNCTION	PASSWORD	NULL_VER...
6	DEFAULT	PASSWORD_LOCK_TIME	PASSWORD	1
7	DEFAULT	PASSWORD_GRACE_TIME	PASSWORD	UNLIMITED
8	DEFAULT	LOGIN_PERIOD	PASSWORD	UNLIMITED

[PASSWORD_REUSE_TIME, PASSWORD_REUSE_MAX 파라미터값 확인]

Step 3) 설정되어 있지 않을 경우 PROFILE 생성 또는 수정 시(ALTER PROFILE) 비밀번호 정책 설정 적용 시
다음 명령어를 사용

CREATE PROFILE prof LIMIT

예시) CREATE PROFILE prof LIMIT

failed_login_attempts 3

password_lock_time 1/1440

password_life_time 90

password_reuse_time unlimited

password_reuse_max 10

password_grace_time 10

password_verify_function verify_function;

D-06 (중)	DBMS > 1. 계정 관리
	DB 사용자 계정을 개별적으로 부여하여 사용
개요	
점검 내용	DB 접근 시 사용자별로 서로 다른 계정을 사용하여 접근하는지 점검
점검 목적	사용자별 별도 DBMS 계정을 사용하여 DB에 접근하는지 점검하여 DB 계정 공유 사용으로 발생할 수 있는 로그 감사 추적 문제를 대비하고자 함
보안 위협	DB 계정을 공유하여 사용할 경우 비인가자의 DB 접근 발생 시 계정 공유 사용으로 인해 로그 감사 추적의 어려움이 발생할 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, MySQL, Altibase, Tiberio, PostgreSQL 등
판단 기준	양호 : 사용자별 계정을 사용하고 있는 경우
	취약 : 공용 계정을 사용하고 있는 경우
조치 방법	사용자별 계정 생성 및 권한 부여
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) 계정 확인(SQL*Plus)

```
SQL> SELECT username FROM dba_users ORDER BY username;
```

Step 2) 공용 계정 삭제

```
SQL> DROP USER '공용 계정';
```

Step 3) 사용자별, 응용 프로그램별 계정 생성

```
SQL> CREATE USER '<계정명>' IDENTIFIED BY '<비밀번호>';
```

Step 4) 권한 부여

```
SQL> GRANT connect, resource TO [계정명];
```

● MSSQL

Step 1) 공용계정 삭제

```
EXEC sp_droplogin '공용 계정';
```


Step 2) 사용자별, 응용 프로그램별 계정 생성

```
CREATE LOGIN '생성 계정' WITH PASSWORD = '비밀번호';
CREATE USER '생성 계정' FOR LOGIN '생성 계정' WITH DEFAULT_SCHEMA = '생성 계정';
ALTER USER '생성 계정';
EXEC sp_adduser '생성 계정', '생성 계정', 'db_owner';
EXEC sp_adduser '생성 계정', '생성 계정', '생성 계정';
EXEC sp_grantdbaccess '생성 계정', '생성 계정';
```

● MySQL

Step 1) 공용용 계정 삭제

```
mysql> DROP USER <계정명>@<호스트명 or IP>
```

Step 2) 사용자별, 응용 프로그램별 계정 생성 및 권한 설정

// 사용자 계정 생성

```
mysql> create user '<계정명>'@<호스트명 or IP>' identified by '비밀번호';
```

// 특정 데이터베이스의 특정 테이블에 select, insert 권한을 부여

```
mysql> grant select, insert on DB이름.테이블명 to '<계정명>'@<호스트명 or IP>;
```

// 특정 데이터베이스의 모든 테이블에 모든 권한을 부여

```
mysql> grant all privileges on DB이름.* to '<계정명>'@<호스트명 or IP>;
```

```
mysql> flush privileges;
```

※ 모든 권한을 부여할 경우, 해당 사용자는 지정된 데이터베이스에서 모든 작업의 수행이 가능하므로 사용자의 관리적 측면에서는 편리하나, 보안적 측면에서는 필요한 최소한의 권한만 부여하여 안정성을 높여야 함

● Altibase

Step 1) DB에 생성된 계정 확인

```
SELECT * FROM system_.sys_users_;
```

Step 2) 공용 계정 확인하여 삭제

```
DROP USER <계정명> CASCADE;
```

Step 3) 사용자별, 응용 프로그램별 등 목적에 맞게 계정 생성

```
CREATE USER <계정명> IDENTIFIED BY <비밀번호>;
```

● Tiberio

Step 1) DB에 생성된 계정 확인

```
SELECT * FROM dba_users_;
```

Step 2) Step1)의 결과에서 공용 계정 확인하여 삭제

```
DROP USER [삭제할 계정] CASCADE;
```

Step 3) 사용자별, 응용 프로그램별 등 목적에 맞게 계정 생성

```
CREATE USER [계정명] IDENTIFIED BY [비밀번호];
```

● PostgreSQL

Step 1) 모든 사용자 확인

```
쿼리문 조회 : SELECT * FROM pg_shadow;
```

```
명령어 조회 : \du
```

Step 2) 불필요 계정 삭제

```
DROP ROLE '삭제할 계정';
```

Step 3) 계정 생성 및 권한 추가

```
CREATE USER '생성할 계정';
```

```
ALTER ROLE '계정명' '권한명' '권한명' .....
```

```
\du (계정 생성 및 권한 확인)
```

※ 계정의 용도 파악 후 불필요한 계정은 삭제, 새로운 계정 생성 시 적절한 권한을 부여하여 생성

D-07 (중)	DBMS > 1. 계정 관리
	root 권한으로 서비스 구동 제한
개요	
점검 내용	서비스 구동 시 root 계정 또는 root 권한으로 구동되는지 점검
점검 목적	root 권한을 제한적으로 사용함으로써 시스템의 손상, 데이터의 유출 및 변조 등을 차단하여 보안 위협을 방지하기 위함
보안 위협	root 권한으로 서비스를 구동할 경우 시스템 손상, 데이터 유출 및 변조, 감사 및 추적의 어려움 등으로 인해 서비스 공격의 표적이 될 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, MySQL, Altibase, Cubrid 등
판단 기준	양호 : DBMS가 root 계정 또는 root 권한이 아닌 별도의 계정 및 권한으로 구동되고 있는 경우
	취약 : DBMS가 root 계정 또는 root 권한으로 구동되고 있는 경우
조치 방법	DBMS 구동 계정 변경
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) 실행 중인 프로세스를 통한 확인

```
$ ps -ef | grep pmon
```

Step 2) Oracle Listener 프로세스 사용자 확인

```
$ ps -ef | grep tnslnr
```

Step 3) 사용자 계정을 'Oracle'로 전환

```
$ su - oracle (현재 사용자/관리자 계정에서 'Oracle' 사용자로 전환)
```

```
$ lsnrctl stop (Oracle Listener 중지)
```

```
$ sqlplus / as sysdba (SQL*Plus 유틸리티를 이용하여 시스템 관리자(SYSDBA) 권한으로 로그인)
```

```
$ shutdown immediate (데이터베이스 종료)
```

Step 4) Oracle 서비스 재시작

```
$ lsnrctl start (Oracle Listener 시작)
```

```
$ sqlplus / as sysdba (SQL*Plus 유틸리티를 이용하여 시스템 관리자(SYSDBA) 권한으로 로그인)
```

\$ startup (데이터베이스 시작)

● MySQL

Step 1) 실행 중인 프로세스를 통한 확인

```
# ps -ef | grep mysqld
```

Step 2) mysql server configuration 파일에서 [mysqld] 그룹의 'user' 지시자의 설정값 확인

```
# cat [mysql server configuration 파일 위치] | grep user (user=mysql로 설정되어 있으면 양호)
```

Step 3) mysql server configuration 파일에서 [mysqld] 그룹의 'user' 지시자 설정

```
# vi [mysql server configuration 파일 위치] (일반적으로 /etc/my.cnf.d/mysql-server.cnf)
```

※ user = [mysqld를 구동할 시스템의 일반 사용자 계정]

● Altibase

Step 1) 실행 중인 프로세스를 통한 확인

```
# ps -ef | grep altibase | grep -v grep
```

Step 2) Altibase 디렉터리 및 파일을 Altibase 전용 계정으로 소유자 변경

```
# chown -R [계정명]:[그룹명] '[Altibase 디렉터리 위치]'
```

Step 3) Altibase 전용 계정으로 DB 구동

● Cubrid

Step 1) 실행 중인 프로세스를 통한 확인

```
# ps -ef | egrep 'cub_master|cub_broker|cub_manager' | grep -v grep
```

(cub_master, cub_broker, cub_manager 데몬이 root 또는 root 권한으로 구동되었는지 확인)

Step 2) 삭제 후 cubrid 계정으로 재설치 또는 같은 DB를 생성 후에 언로드/로드를 통해 기존 데이터 이관

D-08 (상)	DBMS > 1. 계정 관리
	안전한 암호화 알고리즘 사용
개요	
점검 내용	해시 알고리즘 SHA-256 이상의 암호화 알고리즘을 사용하는지 점검
점검 목적	안전한 해시 알고리즘 사용으로 데이터의 기밀성 및 무결성을 보장하고, 사용자 인증을 강화하기 위함
보안 위협	SHA-1이나 MD5와 같은 오래된 알고리즘 사용 시 공격자의 무차별 대입 공격 등으로 비밀번호 유추가 가능하며, 데이터 변조 및 유출의 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, MySQL, Tiberio, PostgreSQL 등
판단 기준	양호 : 해시 알고리즘 SHA-256 이상의 암호화 알고리즘을 사용하고 있는 경우
	취약 : 해시 알고리즘 SHA-256 미만의 암호화 알고리즘을 사용하고 있는 경우
조치 방법	SHA-256 이상의 암호화 알고리즘 적용
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● OracleDB

Step 1) SQL*Plus 쿼리를 통한 암호화 알고리즘 확인

```
SELECT username, password_versions FROM dba_users;
```

Step 2) sqlnet.ora 파일 수정

```
$ vi /u01/app/oracle/product/11.2.0/xe/network/admin/sqlnet.ora
```

```
SQLNET.ALLOWED_LOGON_VERSION_SERVER = 12
```

```
SQLNET.ALLOWED_LOGON_VERSION_CLIENT = 12
```

※ sqlnet.ora 파일 기본 경로

Unix/Linux : \$ORACLE_HOME/network/admin/sqlnet.ora

Windows : %ORACLE_HOME%\network\admin\sqlnet.ora

※ Oracle DB 알고리즘 : 10G(MD5), 11G(SHA-1), 12C(SHA-512, AES)

● MSSQL

Step 1) 저장된 비밀번호 해시 값 확인

```
select name, password_hash from sys.sql_logins;
```

The screenshot shows the SQL Server Enterprise Manager interface. The query window displays the command: `1= select name, password_hash from sys.sql_logins;`. Below the query window, the 'Results' pane shows the output of the query. The results are displayed in a table with two columns: 'name' and 'password_hash'.

	name	password_hash
1	sa	X'0200e10dcb1142499b18328eeb2c9ea0a7794a59bb1c683480ca68adb3fc94346a57bd8b9da510d3a6cdc016fcf8ce7e402bba149190d231e2b9a562c2b12aef1339e2d87b8'
2	##MS_PolicyEventProcessingLogin##	X'020077049d8b0fb2204983b7ed8a079bd3f96d0dcd1e9c2619732104384ca5e6a7902cb004952f424ee03aef743c8ebc8e70d069da9bf727c9ffe11bb0a1ee61b393803215'
3	##MS_PolicyTsqExecutionLogin##	X'02006e5502183c700f5711a81cc3b82f26a8ca4529576d38cc8c95a66b4712bda620452223cc97215714ee1ad74d6977b246c7e139c720f6355ee6a1eba7dcd5cc4e2dc35b0'

※ MSSQL 2012이상에서 사용자 계정의 비밀번호는 32bit Salt를 적용한 SHA-512 해시 알고리즘을 사용

[일반 이용자 패스워드 해시 알고리즘 변경]

Step 1) 데이터베이스 접속

```
USE <데이터베이스명>
```

```
GO
```

Step 1) 열 추가

```
ALTER TABLE <테이블명> ADD <신규 해시 칼럼명> varbinary(256)
```

```
GO
```

Step 2) 새로운 열에 암호화 된 데이터 저장

```
UPDATE <테이블명> SET <신규 해시 칼럼명> = HASHBYTES('SHA2_256', <기존 해시 칼럼명>)
```

```
GO
```

Step 3) 기존 열 제거

```
ALTER TABLE <테이블명> DROP COLUMN <기존 해시 칼럼명>
```

```
GO
```

● MySQL

Step 1) 계정별 암호화 알고리즘 확인

[mysql 5.7]

```
mysql> SELECT user, host, plugin FROM mysql.user; 또는
```

```
mysql> SELECT host, user, plugin, password AS authentication_string FROM mysql.user;
```

[mysql 8.0]

mysql> SELECT user, host, plugin FROM mysql.user; 또는

mysql> SELECT host, user, plugin, authentication_string FROM mysql.user;

Step 2) 비밀번호 및 암호화 알고리즘 설정

[mysql 5.7]

- user 생성 시 적용

CREATE USER '계정명'@'host' IDENTIFIED BY '비밀번호';

- 기존 user 적용

ALTER USER '계정명'@'host' IDENTIFIED '신규 비밀번호';

※ mysql 5.7에서는 기본적으로 mysql_native_password 플러그인이 사용되므로 별도의 지정이 필요하지 않음

[mysql 8.0]

- user 생성 시 적용

mysql> CREATE USER '계정명'@'localhost' IDENTIFIED WITH caching_sha2_password BY '비밀번호';

- 기존 user 적용

mysql> ALTER USER '계정명'@'localhost' IDENTIFIED WITH caching_sha2_password BY '비밀번호';

※ mysql v8.0 이상부터 암호화 알고리즘으로 caching_sha2_password(SHA-256)가 적용됨

※ mysql v5.7 버전에서 사용하던 데이터베이스를 8.0으로 업그레이드하여 mysql_native_password 플러그인이 유지되는 경우 위와 같이 caching_sha2_password 알고리즘을 지정하여 적용할 수 있음

● Tiberio

Step 1) 쿼리문을 통한 암호화 알고리즘 확인

SELECT "TS#", "ENCPTIONALG", "ENCRYPTEDTS" FROM V\$ENCRYPTED_TABLESPACES;

Step 2) 암호화 알고리즘 적용

ALTER TABLESPACE "ENCRYPTED_TS" ENCRYPTION USING '적용할 암호화 알고리즘';

● PostgreSQL

Step 1) psql 접속 후 계정별 암호화 알고리즘 확인

postgres=# SELECT username, passwd FROM pg_shadow;

Step 2) 명령어를 통한 알고리즘 적용

- user 생성 시 적용

```
postgres=# CREATE USER 계정명 password '설정할 비밀번호';
```

- 기존 user 적용

```
postgres=# ALTER USER 계정명 WITH password '설정할 비밀번호';
```

※ default 설정으로 SCRAM-SHA-256 암호화 알고리즘이 적용

※ peer : 로컬에서만 연결이 가능하며 OS에서 클라이언트의 OS 사용자 이름을 얻고 요청한 데이터베이스 사용자 이름과 일치하는지 확인하는 인증 방식

D-09 (중)	DBMS > 1. 계정 관리
	일정 횟수의 로그인 실패 시 이에 대한 잠금정책 설정
개요	
점검 내용	DBMS 설정 중 일정 횟수의 로그인 실패 시 계정 잠금 정책에 대한 설정이 되어있는지 점검
점검 목적	일정 횟수의 로그인 실패 시 계정 잠금 정책을 설정하여 비인가자의 자동화된 무차별 대입 공격, 사전 대입 공격 등을 통한 사용자 계정 비밀번호 유출을 방지하기 위함
보안 위협	일정한 횟수의 로그인 실패 횟수를 설정하여 제한하지 않으면 자동화된 방법으로 계정 및 비밀번호를 획득하여 데이터베이스에 접근하여 정보가 유출될 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, Altibase, Tiberio 등
판단 기준	양호 : 로그인 시도 횟수를 제한하는 값을 설정한 경우
	취약 : 로그인 시도 횟수를 제한하는 값을 설정하지 않은 경우
조치 방법	로그인 시도 횟수 제한 값 설정
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) Failed_login_attempts Profile 파라미터 수정

```
SQL> ALTER PROFILE LIMIT FAILED_LOGIN_ATTEMPTS XX; (XX회 이하로 설정)
```

Step 2) Profile 적용

```
SQL> connect / as sysdba
```

```
SQL> @$Ora_Home/rdbms/admin/utlpwdmg. 또는
```

default profile에 unlimited로 설정하고 이 default 값을 적용하고자 하는 profile에 적용

```
SQL> ALTER PROFILE DEFAULT LIMIT PASSWORD_LOCK_TIME UNLIMITED;
```

해당 Profile이 잠길 경우 자동으로 해제되지 않음

```
SQL> ALTER PROFILE [Profile명] LIMIT PASSWORD_LOCK_TIME XX;
```

해당 Profile이 잠길 경우 XX일 이후 잠금 해제

※ utlpwdmg.sql : 비밀번호 관리 관련 기능 설정 및 Profile 수정 역할을 하는 스크립트

● Altibase

Step 1) `SELECT * FROM system_.sys_users_;`

Step 2) 아래 Property에 대해 비밀번호 정책 설정

CASE_SENSITIVE_PASSWORD
 FAILED_LOGIN_ATTEMPTS
 PASSWORD_LOCK_TIME
 PASSWORD_LIFE_TIME
 PASSWORD_GRACE_TIME
 PASSWORD_REUSE_TIME
 PASSWORD_REUSE_MAX
 PASSWORD_VERIFY_FUNCTION

Step 3) 정책 적용 시 다음 명령어를 사용

ALTER USER 계정명 LIMIT (Property 숫자);
 예시) ALTER USER testuser LIMIT (FAILED_LOGIN_ATTEMPTS 7);

● Tiberio

Step 1) 사용자별 비밀번호 PROFILE 적용 여부 확인

`SELECT * FROM dba_users;`

#	USERNAME	USER_ID	PASSWORD	ACCOUNT_...	LOCK_DATE	EXPIRY_DATE	DEFAULT_...	CREATED	PROFILE	DEFAULT_...
1	SYS	11	p45uM4F6...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP
2	SYSG15	12	ZDn9sWHS...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP
3	OUTLN	13	vNhIe1qd...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
4	TIBERO	18	+N2TQ1DX...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
5	TIBERO1	19	+N2TQ1DX...	OPEN	<NULL>	<NULL>	USR	2024/07/15	<NULL>	TEMP
6	SYS	8	lp7ny7YD...	OPEN	<NULL>	<NULL>	SYSTEM	2024/07/15	<NULL>	TEMP

[사용자별 비밀번호 PROFILE 적용 여부 확인]

Step 2) 설정되어 있을 경우 PROFILE 설정 내용 확인

`SELECT * FROM dba_profiles;`

#	PROFILE	RESOURCE_NAME	RESOURCE_TYPE	LIMIT
1	DEFAULT	FAILED_LOGIN_ATTEMPTS	PASSWORD	UNLIMITED
2	DEFAULT	PASSWORD_LIFE_TIME	PASSWORD	UNLIMITED
3	DEFAULT	PASSWORD_REUSE_TIME	PASSWORD	UNLIMITED
4	DEFAULT	PASSWORD_REUSE_MAX	PASSWORD	UNLIMITED
5	DEFAULT	PASSWORD_VERIFY_FUNCTION	PASSWORD	NULL_VER...
6	DEFAULT	PASSWORD_LOCK_TIME	PASSWORD	1
7	DEFAULT	PASSWORD_GRACE_TIME	PASSWORD	UNLIMITED
8	DEFAULT	LOGIN_PERIOD	PASSWORD	UNLIMITED

[PROFILE 설정 내용 확인]

Step 3) 설정되어 있지 않을 경우 PROFILE 생성 시(또는 수정 시 ALTER PROFILE) 비밀번호 정책 설정 정책 적용 시 다음 명령어를 사용

```
CREATE PROFILE prof LIMIT
```

예시) CREATE PROFILE prof LIMIT

```
FAILED_LOGIN_ATTEMPTS 3
```

```
PASSWORD_LOCK_TIME 1/1440
```

```
PASSWORD_LIFE_TIME 90
```

```
PASSWORD_REUSE_TIME UNLIMITED
```

```
PASSWORD_REUSE_MAX 10
```

```
PASSWORD_GRACE_TIME 10
```

```
PASSWORD_VERIFY_FUNCTION verify_function;
```

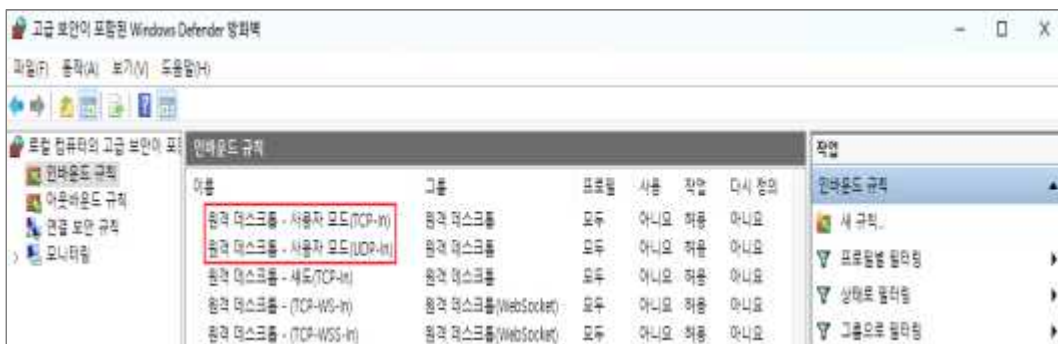
D-10 (상)	DBMS > 2. 접근 관리
	원격에서 DB 서버로의 접속 제한
개요	
점검 내용	지정된 IP주소만 DB 서버에 접근 가능하도록 설정되어 있는지 점검
점검 목적	지정된 IP주소만 DB 서버에 접근 가능하도록 설정되어 있는지 점검하여 비인가자의 DB 서버 접근을 원천적으로 차단하고자 함
보안 위협	DB 서버 접속 시 IP주소 제한이 적용되지 않은 경우 비인가자가 내·외부망 위치에 상관없이 DB 서버에 접근할 수 있는 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Windows OS, Oracle DB, MySQL, Altibase, Tiberio, PostgreSQL 등
판단 기준	양호 : DB 서버에 지정된 IP주소에서만 접근 가능하도록 제한한 경우
	취약 : DB 서버에 지정된 IP주소에서만 접근 가능하도록 제한하지 않은 경우
조치 방법	DB 서버에 대해 지정된 IP주소에서만 접근 가능하도록 설정
조치 시 영향	허용되지 않은 IP에서 접속 제한

점검 및 조치 사례

● Windows OS

Step 1) 특정 IP주소에서만 접속 가능하도록 방화벽 등이 설정되어 있는지 확인

시작 > 제어판 > 시스템 및 보안 > Windows Defender 방화벽 > 고급 설정 > 고급 보안이 포함된 Windows Defender 방화벽 > 인바운드 규칙 > 원격 데스크톱 - 사용자 모드(TCP-In)/사용자 모드(UDP-In)



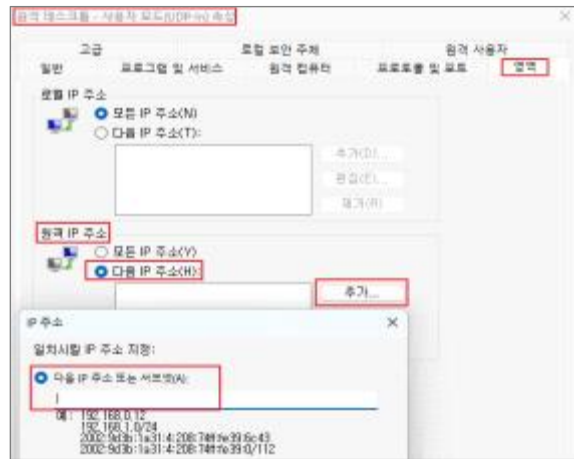
[원격 데스크톱 방화벽 규칙 확인]

Step 2) DB서버에 접근 가능한 특정 IP 지정

원격 데스크톱 - 사용자 모드(TCP-In)/사용자 모드(UDP-In) 속성 > 영역 > 원격 IP주소 > 다음 IP주소 > 추가 > IP주소 입력



[원격 데스크톱 - 사용자 모드(TCP-In) 속성]



[원격 데스크톱 - 사용자 모드(UDP-In) 속성]

● Oracle DB

Step 1) oracle 계정으로 로그인

```
su - oracle
```

Step 1) 텍스트 에디터로 \$ORA_NET/sqlnet.ora 파일 오픈

```
vi $ORA_NET/sqlnet.ora
```

Step 2) sqlnet.ora 파일의 끝에 다음 두 라인을 추가

```
tcp.validnode_checking = yes
```

```
tcp.invited_nodes = ( 127.0.0.1, [allowed IP's] )
```

Step 3) Listener 재시작

```
$ORACLE_HOME/bin/lsnrctl stop
```

```
$ORACLE_HOME/bin/lsnrctl start
```

● MySQL

Step 1) user 테이블을 조회하여 모든 클라이언트에서 접속 가능하도록 설정되어 있는 계정을 특정 IP에서만 접속 가능하도록 변경

```
mysql> UPDATE user SET host = '<접속 IP>' WHERE user = '<계정명>' and host='%';
```

● Altibase

Step 1) Altibase HDB Property 파일을 수정하여 접근 제어 적용

\$Altibase_HOME/conf/altibase.properties의 IP Access Control Lists에서 내부 정책에 맞게 수정

※ Altibase HDB 서버가 실행되지 않은 상태에서 할 수 있는 정적인 환경설정 방법으로 Property 파일에서 해당 구성 요소를 특정 값으로 설정한 후 Altibase HDB 서버를 구동해야 수정된 값이 Altibase HDB 서버에 반영 됨

```
# examples of IPv4 addresses.
#access_list = deny      ,0.0.0.0          ,0.0.0.0
#access_list = permit    ,192.168.3.0      ,255.255.255.0
#access_list = permit    ,192.168.1.0      ,255.255.255.0
#access_list = permit    ,192.168.1.131    ,255.255.255.255
```

[altibase.properties 파일]

Step 2) Altibase HDB 서버에 적용된 접근 제어 설정 확인

iSQL> SELECT name, value1 FROM v\$sqlproperty WHERE name LIKE 'ACCESS_CONTROL_%';

● PostgreSQL

Step 1) Data 디렉터리 내에 postgres.conf 파일 설정

```
#-----
# CONNECTIONS AND AUTHENTICATION
#-----
# - Connection Settings -
listen_addresses = 'localhost'      # what IP address(es) to listen on;
                                     # comma-separated list of addresses;
```

[postgres.conf 파일]

※ listen_addresses는 서버가 클라이언트 애플리케이션의 연결을 수신 대기할 TCP/IP 주소를 지정함. 호스트 이름 및/또는 숫자 IP 주소의 쉼표로 구분된 목록 형식으로 지정할 수 있으며, *를 사용하는 경우 모든 IP에 대해 수신 대기함. 기본값은 localhost 이며 로컬 TCP/IP “루프백” 연결 만 허용

Step 2) Data 디렉터리 내에 pg_hba.conf 파일 설정

TYPE	DATABASE	USER	CIDR-ADDRESS	METHOD
-----	-----	-----	-----	-----
host	(DB명)	(사용자)	(접속 허용 IP)	md5

#	TYPE	DATABASE	USER	ADDRESS	METHOD
# "local" is for the database's internal connections only.					
local	all		all		trust
# IP for local connections:					
host	all		all	127.0.0.1/32	scram-sha-256
# IP for local connections:					
host	all		all	:::1/128	scram-sha-256
# Allow replication connections from localhost, by a user with the					
# replication privileges.					
local	replication		all		trust
host	replication		all	127.0.0.1/32	scram-sha-256
host	replication		all	:::1/128	scram-sha-256

[pg_hba.conf 파일]

Step 3) USER에 접근 허용 '계정명'과 CIDR-ADDRESS에 접속을 '허용할 IP' 설정

※ PostgreSQL은 기본 설치 시 외부에서 접속할 수 없음

※ IP 접근 제한 설정 시 postgresql.conf와 pg_hba.conf 두 개의 설정 파일이 연계되어 있으므로 하나의 파일이라도 설정이 잘못되어 있는 경우 DB 접속이 불가능 할 수 있음.

● Tibero

Step 1) LSNR_INVITED_IP(특정 IP주소를 갖는 클라이언트만 허용, 그 외 차단)

\$TB_SID.tip 파일 안에 다음 예시 내용을 참조하여 입력

LSNR_INVITED_IP=192.168.1.1;192.168.2.0/24;192.1.0.0/16

LSNR_INVITED_IP의 최대 길이는 255자이며, 256자 이상의 IP주소를 설정할 경우에는 LSNR_INVITED_IP_FILE을 사용

(특정 IP주소를 기재한 파일의 절대 경로를 LSNR_INVITED_IP_FILE에 입력)

※ 초기화 파라미터에 설정된 IP주소에 따라 클라이언트의 네트워크 접속을 허용하거나 차단

※ \$TB_SID는 Tibero 설치 시 입력한 데이터베이스 이름과 동일(/ c:/tibero/tiberos5/config/데이터베이스.tip)

- \$TB_SID.tip 파일에 LSNR_INVITED_IP와 LSNR_DENIED_IP가 모두 설정되어 있는 경우 LSNR_DENIED_IP의 설정은 무시되며 LSNR_INVITED_IP만 적용된다. 즉, LSNR_INVITED_IP에 설정된 IP 주소의 클라이언트를 제외하고는 모든 접속이 차단된다.
- \$TB_SID.tip 파일에 LSNR_INVITED_IP와 LSNR_DENIED_IP가 모두 설정되지 않은 경우 모든 클라이언트의 네트워크 접속이 허용된다.
- 루프백 주소(Loopback Address, 127.0.0.1)에서 접속하는 경우 LSNR_INVITED_IP 또는 LSNR_DENIED_IP의 설정과는 무관하게 항상 허용된다. X Tibero 서버를 운영하는 중에 서버를 다시 기동하지 않고 LSNR_INVITED_IP 또는 LSNR_DENIED_IP의 설정을 변경하려는 경우 우선 \$TB_SID.tip 파일에 LSNR_INVITED_IP 또는 LSNR_DENIED_IP의 설정을 변경한 후 파일을 저장하고 다음의 명령을 실행한다.

ALTER SYSTEM LISTENER PARAMETER RELOAD;

위의 명령을 실행하면 \$TB_SID.tip 파일에서 LSNR_INVITED_IP 또는 LSNR_DENIED_IP의 내용을 다시 읽어 변경된 내용을 실시간으로 적용한다.

D-11 (상)	DBMS > 2. 접근 관리
	DBA 이외의 인가되지 않은 사용자가 시스템 테이블에 접근할 수 없도록 설정
개요	
점검 내용	시스템 테이블에 일반 사용자 계정이 접근할 수 없도록 설정되어 있는지 점검
점검 목적	시스템 테이블의 일반 사용자 계정 접근 제한 설정 적용 여부를 점검하여 일반 사용자 계정 유출 시 발생할 수 있는 비인가자의 시스템 테이블 접근 위험을 차단하기 위함
보안 위협	시스템 테이블의 일반 사용자 계정 접근 제한 설정이 되어있지 않을 경우 Object, 사용자, 테이블 및 뷰, 작업 내역 등의 시스템 테이블에 저장된 정보가 누출될 수 있음
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, MySQL, Altibase, Tiberio, PostgreSQL 등
판단 기준	양호 : 시스템 테이블에 DBA만 접근 가능하도록 설정되어 있는 경우
	취약 : 시스템 테이블에 DBA 외 일반 사용자 계정이 접근 가능하도록 설정되어 있는 경우
조치 방법	시스템 테이블에 일반 사용자 계정이 접근할 수 없도록 설정
조치 시 영향	일반 계정으로 시스템 테이블 접근 불가

점검 및 조치 사례

● Oracle DB, Tiberio

Step 1) DBA만 접근 가능한 테이블의 권한 확인(SQL*Plus)

```
SQL> SELECT grantee, privilege, owner, table_name FROM dba_tab_privs WHERE (owner = 'SYS' or table_name LIKE 'DBA_%') AND privilege <> 'EXECUTE' AND grantee NOT IN ('PUBLIC', 'AQ_ADMINISTRATOR_ROLE', 'AQ_USER_ROLE', 'AURORA$JIS$UTILITY$', 'OSE$HTTP$ADMIN', 'TRACESVR', 'CTXSYS', 'DBA', 'DELETE_CATALOG_ROLE', 'EXECUTE_CATALOG_ROLE', 'EXP_FULL_DATABASE', 'GATHER_SYSTEM_STATISTICS', 'HS_ADMIN_ROLE', 'IMP_FULL_DATABASE', 'LOGSTDBY_ADMINISTRATOR', 'MDSYS', 'ODM', 'OEM_MONITOR', 'OLAPSYS', 'ORDSYS', 'OUTLN', 'RECOVERY_CATALOG_OWNER', 'SELECT_CATALOG_ROLE', 'SNMPAGENT', 'SYSTEM', 'WKSYS', 'WKUSER', 'WMSYS', 'WM_ADMIN_ROLE', 'XDB', 'LBACSYS', 'PERFSTAT', 'XDBADMIN') AND grantee NOT IN (SELECT grantee FROM dba_role_privs WHERE granted_role = 'DBA') ORDER BY grantee; (어떤 계정이나 role이 나타나지 않으면 양호)
```

Step 2) 불필요하게 테이블 접근 권한이 사용자 계정에 할당된 경우(SQL*Plus)

```
SQL> REVOKE <권한> ON <Object> FROM user;
```

● MSSQL

Step 1) system tables 접근 권한이 PUBLIC, GUEST 또는 비인가된 사용자에게 부여된 경우 접근 권한을 제거

```
REVOKE <권한> ON <Object> FROM [계정명][PUBLIC][GUEST];
```

Step 2) 시스템 테이블에 접근하기 위해서는 stored procedure 또는 information_schema views를 통해 접근해야 함

Step 3) 시스템 테이블에 접근 가능한 stored procedure는 사용이 제한되어야 함

● MySQL

Step 1) 사용자 계정에 부여된 권한 확인

```
SHOW GRANTS FOR <계정명>;
```

Step 2) 접근이 필요한 데이터베이스 및 테이블에만 권한 적용

```
GRANT <권한> privileges ON <DB명>.<테이블명> to '<계정명>'@<호스트명 or IP>;
```

● Altibase

Step 1) sys_tables_를 조회하여 system_ 외 접근 계정 유무 확인

```
SELECT * FROM system_.sys_tables_;
```

Step 2) 불필요 계정 접근 시 해당 접근 해제

● PostgreSQL

Step 1) 사용자 및 역할 권한 정보 조회

```
SELECT * FROM information_schema.role_table_grants;
```

Step 2) 스키마명에 해당되는 Table에 대한 접근 권한을 일반 사용자로부터 제거

```
REVOKE [all,select,insert,update...] ON all tables IN schema '스키마명' FROM '계정명';
```

D-12 (상)	DBMS > 2. 접근 관리
	안전한 리스너 비밀번호 설정 및 사용
개요	
점검 내용	오라클 데이터베이스 Listener의 비밀번호 설정 여부 점검
점검 목적	Listener의 Owner는 DBA가 아니더라도 Listener를 shutdown 시키거나 DB 서버에 임의의 파일을 생성할 수 있으며, 원격에서 LSNRCTL 유틸리티를 사용하여 listener.ora 파일에 대한 변경이 가능하므로 Listener에 비밀번호를 설정하여 비인가자가 이를 수정하지 못하도록 하기 위함
보안 위협	Listener에 비밀번호가 설정되지 않았을 경우 DoS, 정보 획득, Listener 프로세스를 중지시킬 수 있는 위험이 존재함
참고	※ 오라클 Listener : 클라이언트가 원격에서 오라클 DB에 접근할 때 접근 요청을 처리하기 위한 서버 쪽 프로세스, 혹은 네트워크 인터페이스를 말하며 TCP/1521 포트를 사용함 ※ listener.ora : 오라클 서버에서 클라이언트의 요청을 듣고, 클라이언트와의 통신 환경을 설정하는 파일
점검 대상 및 판단 기준	
대상	Oracle DB
판단 기준	양호 : Listener의 비밀번호가 설정된 경우
	취약 : Listener의 비밀번호가 설정되어 있지 않은 경우
조치 방법	Listener 비밀번호 설정
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) Listener 비밀번호 설정

```
LSNRCTL> change_password
```

```
Old password: <Old Password> (Not displayed)
```

```
New password: <New password> (Not displayed)
```

```
Reenter new password: <New password> (Not displayed)Connecting (DESCRIPTION=(ADDRESS=(PROTOCOL=TCP)(HOST=prolin1)(PORT=1521)(IP=FIRST)))
```

```
Password change for LISTENER
```

```
The command completed successfully
```

```
LSNRCTL> set password
```

```
LSNRCTL> save_config
```

Step 2) Listener 매개변수 설정

1. \$TNS_ADMIN/listener.ora 파일 내에 아래 Option 추가

```
PASSWORDS_<listener_name> = <Encrypted Password>
```

```
ADMIN_RESTRICTIONS_<listener_name> = ON
```

2. Listener 재시작 및 상태 확인

```
LSNRCTL> lsnrctl stop
```

```
LSNRCTL> lsnrctl start
```

```
LSNRCTL> lsnrctl reload
```

```
LSNRCTL> lsnrctl status
```

※ Oracle 12c release 2 이후 버전은 Listener 비밀번호 설정을 지원하지 않으므로 해당사항 없음

D-13 (중)	DBMS > 2. 접근 관리
	불필요한 ODBC/OLE-DB 데이터 소스와 드라이버를 제거하여 사용
개요	
점검 내용	사용하지 않는 불필요한 ODBC/OLE-DB가 설치되어 있는지 점검
점검 목적	불필요한 데이터 소스 및 드라이버를 제거함으로써 비인가자에 의한 데이터베이스 접속 및 자료 유출을 차단하기 위함
보안 위협	불필요한 ODBC/OLE-DB 데이터 소스를 통한 비인가자의 데이터베이스 접속 및 주요 정보유출에 대한 위험이 발생할 수 있음
참고	※ 특정 샘플 응용 프로그램은 샘플 데이터베이스를 위해 ODBC 데이터 소스를 설치하거나 불필요한 ODBC/OLE-DB 데이터베이스 드라이버를 설치하므로 불필요한 데이터 소스나 드라이버는 ODBC 데이터 소스 관리자 도구를 이용해서 제거하는 것이 바람직함
점검 대상 및 판단 기준	
대상	Windows OS
판단 기준	양호 : 불필요한 ODBC/OLE-DB가 설치되지 않은 경우
	취약 : 불필요한 ODBC/OLE-DB가 설치된 경우
조치 방법	불필요한 ODBC/OLE-DB 제거
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Windows NT

Step 1) 사용하지 않는 불필요한 ODBC 데이터 소스 확인

시작> 설정 > 제어판> 관리 도구> 데이터 원본(ODBC)> 시스템 DSN

Step 2) 사용하지 않는 데이터 소스 제거

● Windows 2000, 2003, 2008, 2012, 2016

Step 1) 사용하지 않는 불필요한 ODBC 데이터 소스 확인

시작> 설정 > 제어판> 관리 도구> 데이터 원본(ODBC)> 시스템 DSN> 해당 드라이브 클릭

Step 2) 사용하지 않는 데이터 소스 제거



[사용하지 않는 데이터 소스 제거]

● Windows 2019, 2022

Step 1) ODBC 사용하지 않는 불필요한 데이터 소스 확인

시작 > 설정 > 제어판 > 시스템 및 보안 > 관리 도구 > ODBC 데이터 원본 관리자(32비트/64비트) > 시스템DSN > 해당 드라이브 클릭

Step 2) 사용하지 않는 데이터 소스 제거



[사용하지 않는 데이터 소스 제거]

D-14 (중)	DBMS > 2. 접근 관리
	데이터베이스의 주요 설정 파일, 비밀번호 파일 등과 같은 주요 파일들의 접근 권한이 적절하게 설정
개요	
점검 내용	데이터베이스의 주요 파일들에 대해 관리자를 제외한 일반 사용자의 파일 수정 권한을 제거하였는지 점검
점검 목적	데이터베이스의 주요 파일에 관리자를 제외한 일반 사용자의 파일 수정 권한을 제거함으로써 비인가자에 의한 DBMS 주요 파일 변경이나 삭제를 방지하고 주요 정보 유출을 방지할 수 있음
보안 위협	데이터베이스 주요 파일에 비인가자가 접근하여 수정 및 삭제 시 데이터베이스 운영에 장애가 발생할 수 있으며 계정 비밀번호 정보 등 중요 정보의 유출 위험이 존재함
참고	※ 데이터베이스의 주요 파일: orapw.ora, listener.ora, init<SID>.ora, redo 파일, 데이터베이스 설정 파일, 네트워크 설정 파일 등
점검 대상 및 판단 기준	
대상	Oracle DB, PostgreSQL, Cubrid 등
판단 기준	양호 : 주요 설정 파일 및 디렉터리의 권한 설정 시 일반 사용자의 수정 권한을 제거한 경우
	취약 : 주요 설정 파일 및 디렉터리의 권한 설정 시 일반 사용자의 수정 권한을 제거하지 않은 경우
조치 방법	주요 설정 파일 및 디렉터리의 권한 설정 변경
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

[Unix OS]

Step 1) 디렉터리 또는 파일의 권한 점검

```
$ORACLE_HOME/bin/oracle (755)
```

```
$ORACLE_HOME/bin/[ ] (755)
```

```
→ [.sqlplus,sqlldr,sqlload,proc,oraenv,oerr,exp,imp,tkprof,tnsping,wrap]
```

```
$ORACLE_HOME/bin/[ ] (750)
```

```
→ [.svrmgrl, lsnrctl, dbsnmp]
```

```
$ORACLE_HOME/network (755)
```

```
$ORACLE_HOME/network/admin/[ ] (755)
```

```
→ [.listener.ora, sqlnet.ora 등]
```

```
$ORACLE_HOME/lib (755)
```

```

$ORACLE_HOME/network/admin/[ ] (644)
→ [.tnsnames.ora, protocol.ora, sqlpnet.ora]
$ORACLE_HOME/dbs/init.ora (640)
$ORACLE_HOME/dbs/init<SID>.ora (640)
- Find $ORACLE_HOME -name init*.ora -print
- 파일 및 디렉터리의 권한 설정 변경
# chmod <적용 권한> <파일명>

```

Step 2) redo 파일, 데이터베이스 설정 파일, 데이터 파일 위치 확인(SQL*Plus)

```

SQL> Select value from v$parameter where name='spfile';
SQL> Select 'Control Files: '||value from v$parameter where name='control_files';
SQL> select 'Control Files: '||value from v$parameter where name='spfile';
SQL> select 'Logfile: '||member from v$logfile;
SQL> select 'Datafile: '||name from v$datafile;
- 파일 및 디렉터리의 권한 설정 변경
# chmod <적용 권한> <파일명>

```

[Windows OS]

Step 1) 패스워드 파일(orapw<SID>) 접근 권한은 administrators, system group, owner group, oracle service count, DBA에게 모든 권한 또는, 그 이하로 설정하고 다른 그룹은 제거함

● MySQL

[Unix OS]

Step 1) 설정 파일 (my.cnf, my.ini)의 접근 권한 설정
 설정 파일 (my.cnf, my.ini)의 접근 권한을 설정 파일에 대한 보호를 위하여 600 또는 640으로 설정
 my.cnf 파일 위치: /etc/my.cnf, <각 홈디렉터리>/my.cnf
 # chmod 600 ./my.cnf

[Windows OS]

Step 1) 설정 파일의 접근 권한은 Administrators, SYSTEM, Owner에게 모든 권한 또는 그 이하로 설정하고 다른 그룹은 제거함

● PostgreSQL

[Unix OS]

Step 1) 주요 설정 파일 위치 확인

postgresql.conf 파일 위치: [\$datadir]

DB 접속 통제 설정 파일 위치: /postgres/data/pg_hba.conf, /postgres/data/pg_ident.conf

log_directory : /log_directory/pg_log

Step 2) 주요 설정 파일의 권한 설정

환경설정 파일(postgresql.conf)의 권한을 640 이하로 설정

```
# chmod 640 [$datadir]/postgresql.conf
```

DB접속 통제 설정 파일(pg_hba.conf, pg_ident.conf)의 권한을 640 이하로 설정

```
# chmod 640 ./pg_hba.conf
```

```
# chmod 640 ./pg_ident.conf
```

히스토리 파일 (.psql_history)의 권한을 600 이하로 설정

```
$chmod 600 .psql_history
```

Log 파일(pg_log)의 권한을 640 이하로 설정

```
#chmod 640 [Log 파일]
```

[Windows OS]

Step 1) 주요 환경설정 파일의 접근 권한은 Administrators, SYSTEM, Owner에게 모든 권한 또는 필요 권한만 부여하여 설정하고 기타 다른 그룹은 권한 제거

● Cubrid

Step 1) cubrid.conf 파일 권한 확인

```
# ls -l $_CUBRID_DATABASES/conf/cubrid.conf 또는
```

```
# ls -l /root/CUBRID-11.2.8.0824-bf70ab7-Linux.x86_64/conf/cubrid.conf (설치경로 직접 입력)
```

Step 2) cubrid.conf 파일 권한 설정

```
# sudo chmod 640 /root/CUBRID-11.2.8.0824-bf70ab7-Linux.x86_64/conf/cubrid.conf
```

※ cubrid.conf 파일의 권한이 600 또는 640일 경우 양호함

D-15 (하)	DBMS > 2. 접근 관리
	관리자 이외의 사용자가 오라클 리스너의 접속을 통해 리스너 로그 및 trace 파일에 대한 변경 제한
개요	
점검 내용	Listener 관련 설정 파일의 접근 권한을 관리자만 가능하게 하고 Listener 파라미터의 변경 방지에 대한 옵션 설정 여부 점검
점검 목적	Listener 설정 파일 및 파라미터 변경 방지 옵션을 설정하여 비인가자의 Listener를 이용한 파라미터 변경을 방지하여 trace 파일 및 Listener 로그의 신뢰도를 유지하기 위함
보안 위협	비인가자가 Oracle의 LSNRCTL 유틸리티를 이용하여 Listener에 직접 접근할 경우, 명령어를 통해 Listener의 모든 파라미터를 변경할 수 있으며, 이로 인해 trace 파일이나 Listener 로그 파일을 변경할 위험이 존재함
참고	※ trace 파일 : 데이터베이스에 문제가 발생했을 시 문제를 진단하고 디버깅할 수 있도록 다양한 정보를 제공하는 파일
점검 대상 및 판단 기준	
대상	Oracle DB
판단 기준	양호 : Listener 관련 설정 파일에 대한 권한이 관리자로 설정되어 있으며, Listener로 파라미터를 변경할 수 없게 옵션이 설정된 경우
	취약 : Listener 관련 설정 파일에 대한 권한이 일반 사용자로 설정되어 있고, Listener로 파라미터를 변경할 수 없게 옵션이 설정되지 않은 경우
조치 방법	주요 파일 및 로그 파일에 대한 권한을 관리자로 제한
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) 파일 권한 확인

\$ORACLE_HOME/network/admin 디렉터리 권한 확인

Unix 계열 : ls -a

Windows 계열 : 파일 속성

LSNRCTL> status ListenerName (listener.ora 파일 확인)

listener.ora 파일에서 ADMIN_RESTRICTIONS_<listener name> = ON 설정 확인

Step 2) ADMIN_RESTRICTIONS_LISTENER 설정 추가

listener.ora 파일에 ADMIN_RESTRICTIONS_<listener name>=ON 라인을 추가한 후 listener를 재실행 하거나 lsnrctl reload 명령어를 실행하여 재로딩

```
# listener.ora Network Configuration File:
SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (SID_NAME = PLSExtProc)
      (ORACLE_HOME = /u01/app/oracle/product/11.2.0/xe)
      (PROGRAM = extproc)
    )
  )

LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC_FOR_XE))
      (ADDRESS = (PROTOCOL = TCP)(HOST = oracle-virtual-machine)(PORT = 1521))
    )
  )

DEFAULT_SERVICE_LISTENER = (XE)

#---ADDED BY TNSLSNR 29-JUL-2024 15:40:51---
PASSWORDS_LISTENER = 0185249A0838B44B
ADMIN_RESTRICTIONS_LISTENER = ON
```

[ADMIN_RESTRICTIONS_<listener name>=ON 옵션 설정]

D-16 (하)	DBMS > 2. 접근 관리
	Windows 인증 모드 사용
개요	
점검 내용	DB 로그인 시 Windows 인증 모드 적절성 점검
점검 목적	적절한 Windows 인증 모드를 적용하여 적합한 복잡성 수준을 유지하기 위함
보안 위협	혼합 인증 모드를 사용하고 sa 계정이 활성화되어 있는 경우, 잘 알려진 sa 계정에 대한 계정 추측 공격의 위험이 존재함
참고	※ 데이터베이스 엔진 인증 모드에는 Windows 인증 모드와 SQL Sever가 있는 혼합 모드 두 가지 구성이 있음. Windows 인증 모드 선택 시 SQL Sever 인증을 위해서 설치 프로그램은 sa라는 비활성화된 계정을 생성하고, 이 계정은 혼합 모드를 사용함으로써 활성화됨. sa 계정은 일반 사용자들에게 잘 알려진 만큼 쉽게 공격의 대상이 될 수 있으므로 꼭 필요하지 않은 경우 비활성화하고, 활성화해야 할 경우 강력한 암호 체계를 사용해야 함 ※ Windows 인증은 kerberos 보안 프로토콜을 사용하며, 강력한 암호 정책을 적용하여 적합한 복잡성 수준을 유지함. 또한, 계정 잠금 및 암호만료를 지원하고 SQL 서버가 Windows에서 제공하는 자격 증명을 신뢰한 트러스트 연결을 사용하기 때문에 Windows 인증 모드 사용을 권고함 ※ sa 계정: 데이터베이스 서버 설치 시 자동으로 생성되며 DB 서버 관리자 계정 ※ kerberos 보안 프로토콜: 개방된 컴퓨터 네트워크 내에서 서비스 요구를 인증하기 위한 보안 시스템
점검 대상 및 판단 기준	
대상	MSSQL
판단 기준	양호 : Windows 인증 모드를 사용하고 sa 계정이 비활성화되어 있는 경우 sa 계정 활성화 시 강력한 암호 정책을 설정한 경우
	취약 : 혼합 인증 모드를 사용하고, 활성화된 sa 계정에 대한 강력한 암호 정책 설정을 하지 않은 경우
조치 방법	Windows 인증 모드 사용
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● MSSQL

Step 1) Windows 인증 모드 활성화

SQL Server Management Studio > 해당 서버 우클릭 > 속성 > 보안 > 서버 인증> Windows 인증 모드 (W)를 클릭하여 활성화



[Windows 인증 모드(W) 활성화]

D-17 (하)	DBMS > 3.옵션 관리
	Audit Table은 데이터베이스 관리자 계정으로 접근하도록 제한
개요	
점검 내용	Audit Table 접근 권한이 관리자 계정으로 제한되고 있는지 점검
점검 목적	Audit Table 접근 권한을 관리자 계정으로 제한함으로써 비인가자가 감사 데이터의 수정, 삭제하는 것을 방지하고, 감사 기록의 무결성과 신뢰성을 보장하기 위함
보안 위협	Audit Table이 데이터베이스 관리자 계정에 속하지 않을 경우, 비인가자가 감사 데이터의 수정, 삭제 등을 수행할 수 있으므로 보안 사고 발생 시 원인 분석이 불가능하게 되며, 이로 인해 재발 방지를 위한 조치를 할 수 없으므로 동일 유형의 공격이 반복되거나 시스템 취약점의 악용이 반복될 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, Altibase, Tiberio 등
판단 기준	양호 : Audit Table 접근 권한이 관리자 계정으로 설정한 경우
	취약 : Audit Table 접근 권한이 일반 계정으로 설정한 경우
조치 방법	Audit Table 접근 권한을 관리자 계정으로 제한
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB, Tiberio

Step 1) 설정 확인(SQL*Plus)

```
SQL> SELECT owner FROM dba_tables WHERE table_name = 'AUD$';
```

Step 2) Audit table에 접근할 권한이 없는 계정이 확인될 경우 권한 삭제(SYS 또는 SYSTEM 제외)

```
REVOKE <권한> ON <Object> FROM <계정명>;
```

● Altibase

Step 1) 사용자 계정을 조회하여 SYSTEM_, SYS의 USER_ID를 확인

```
SELECT * FROM system_.sys_users_;
```

Step 2) 시스템 테이블 조회 내용 중 AUDIT 관련 테이블 정보의 TABLE_ID 확인

```
SELECT * FROM system_.sys_tables_;
```

Step 3) Audit Table의 계정에 불필요한 권한이 부여되어 있는 경우 권한 삭제

REVOKE <권한> ON [audit_table] FROM [계정명];

● Tiberio

감사 기록은 \$TB_SID.tip 파일에 설정된 AUDIT_TRAIL 파라미터에 따라 데이터베이스 내부 또는 OS 파일에 저장할 수 있으며, OS 파일에 감사 기록을 저장하는 경우 파일의 위치와 최대 크기를 각각 \$TB_SID.tip파일의 AUDIT_FILE_DEST 파라미터와 AUDIT_FILE_SIZE 파라미터로 설정할 수 있음

Step 1) <\$TB_SID.tip> 파일에 아래 내용 입력

AUDIT_TRAIL=DB_EXTENDED

감사 기록에 포함되는 기본 정보 및 사용자가 실행한 SQL 문장까지 저장

※ 다음 정적뷰를 통해 감사 기록 조회가 가능

DBA_AUDIT_TRAIL (SELECT * FROM dba_audit_trail;)

USER_AUDIT_TRAIL (SELECT * FROM user_audit_trail;)

또는

AUDIT_TRAIL=OS

AUDIT_FILE_DEST=/home/Tibero/audit/audit_trail.log

AUDIT_FILE_SIZE=10M

위와 같이 설정하면 "/home/Tibero/audit/audit_trail.log"에 최대 10MB의 크기로 감사 기록이 저장됨

※ 감사 파일이 있는 디렉터리에는 일반 사용자는 접근할 수 없도록 설정

D-18 (상)	DBMS > 3. 옵션 관리
	응용프로그램 또는 DBA 계정의 Role이 Public으로 설정되지 않도록 조정
개요	
점검 내용	응용 프로그램 또는 DBA 계정의 Role이 Public으로 설정되어 있는지 점검
점검 목적	응용 프로그램 또는 DBA 계정의 Role을 점검하여 일반 계정으로 응용 프로그램 테이블이나 DBA 테이블의 접근을 차단하기 위함
보안 위협	응용 프로그램 또는 DBA 계정의 Role이 Public으로 설정된 경우 일반 계정에서도 응용 프로그램 테이블 및 DBA 테이블로 접근할 수 있으므로 중요 정보 유출의 위험이 존재함
참고	※ Role : 사용자에게 허가할 수 있는 권한들의 집합
점검 대상 및 판단 기준	
대상	Oracle DB, Altibase, Tiber, Cubrid 등
판단 기준	양호 : DBA 계정의 Role이 Public으로 설정되지 않은 경우
	취약 : DBA 계정의 Role이 Public으로 설정된 경우
조치 방법	DBA 계정의 Role 설정에서 Public 그룹 권한 취소
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) DBA Role 설정 확인(SQL*Plus)

```
SQL> SELECT granted_role FROM dba_role_privs WHERE grantee = 'PUBLIC';
```

Step 2) PUBLIC 그룹의 권한 취소(SQL*Plus)

```
SQL> REVOKE [Role name] FROM PUBLIC;
```

● Altibase

Step 1) 사용자 정보를 조회하여 Object 권한, 시스템 권한이 Public 또는 Guest에게 부여되어 있는지 확인

```
SELECT * FROM system_.sys_users_;
```

```
SELECT * FROM system_.sys_grant_object_;
```

```
SELECT * FROM system_.sys_grant_system_;
```

GRANTOR_ID : 권한을 부여한 사용자의 식별자로, SYS_USERS_ 메타테이블의 한 USER_ID 값과 동일

GRANTEE_ID : 권한을 부여받은 사용자의 식별자로, SYS_USERS_ 메타테이블의 한 USER_ID 값과

동일함. 단, Object 권한을 Public에게 부여한 경우, SYS_USERS_메타테이블에 존재하지 않는 USER_ID 값인 "0"이 칼럼에 나타남

Step 2) 불필요 권한 회수

REVOKE <권한> ON <Object> FROM [계정명];

● Tiberio

Step 1) 사용자 정보를 조회하여 Role 부여가 적절한지 확인

SELECT * FROM dba_role_privs;

SELECT * FROM user_role_privs;

Step 2) 불필요 권한 회수

REVOKE <권한> FROM [계정명];

D-19 (상)	DBMS > 3. 옵션 관리
	OS_ROLES, REMOTE_OS_AUTHENTICATION, REMOTE_OS_ROLES를 FALSE로 설정
개요	
점검 내용	OS_ROLES, REMOTE_OS_AUTHENTICATION, REMOTE_OS_ROLES가 FALSE로 설정이 적용되어 있는지 점검
점검 목적	OS_ROLES, REMOTE_OS_AUTHENTICATION, REMOTE_OS_ROLES의 설정을 점검하여 비인가자들의 데이터베이스 접근을 막고 데이터베이스 관리자에 의한 사용자 Role 설정이 가능하게 하기 위함
보안 위협	- OS_ROLES가 TRUE로 설정된 경우, 데이터베이스 접근 제어로 컨트롤되지 않는 OS 그룹에 의해 GRANT 된 권한이 허락되어 악의적인 사용자가 시스템 권한을 악용할 위험이 존재 - REMOTE_OS_ROLES가 TRUE로 설정된 경우, 원격 사용자가 OS의 다른 사용자로 속여 데이터베이스에 접근할 수 있으므로 중요 정보에 대한 무단 접근 및 권한 상승의 위험이 존재함 - REMOTE_OS_AUTHENT가 TRUE로 설정된 경우, 신뢰하는 원격 호스트에서 인증 절차 없이 데이터베이스에 접속할 수 있으므로 중요 정보의 유출 위험이 존재함
참고	※ OS_ROLES: OS 그룹에 의한 사용자의 롤 부여를 가능하게 하도록 설정 ※ REMOTE_OS_AUTHENT: 원격지의 OS 인증 허용 여부를 설정 ※ REMOTE_OS_ROLES: OS가 원격 클라이언트에 대한 롤의 지정이 가능하도록 설정
점검 대상 및 판단 기준	
대상	Oracle DB
판단 기준	양호 : OS_ROLES, REMOTE_OS_AUTHENTICATION, REMOTE_OS_ROLES 설정이 FALSE로 설정된 경우
	취약 : OS_ROLES, REMOTE_OS_AUTHENTICATION, REMOTE_OS_ROLES 설정이 TRUE로 설정되지 않은 경우
조치 방법	OS_ROLES, REMOTE_OS_AUTHENTICATION, REMOTE_OS_ROLES 설정을 FALSE로 변경
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) OS_ROLES 확인

```
SQL> SHOW PARAMETER os_roles;
```

```
SQL> SELECT value FROM v$parameter WHERE name = 'os_roles';
```

Oracle_HomeDirectory/admin/pfile/init.ora에서 OS_ROLE = FALSE 추가 및 인스턴스 재시작

Step 2) REMOTE_OS_AUTHENTICATION 확인

```
SQL> SHOW PARAMETER remote_os_authent;
```

```
SQL> SELECT value FROM v$parameter WHERE name = 'remote_os_authent';
```

Oracle_HomeDirectory/admin/pfile/init.ora에서 remote_os_authent = FALSE 추가 및 인스턴스 재시작

Step 3) REMOTE_OS_ROLES 확인

```
SQL> SHOW PARAMETER remote_os_roles;
```

```
SQL> SELECT value FROM v$parameter WHERE name = 'remote_os_roles';
```

Oracle_HomeDirectory/admin/pfile/init.ora에서 remote_os_roles = FALSE 추가 및 인스턴스 재시작

※ 버전 9i 이후 버전은 SPFILE을 재생성해야 하므로, DBMS를 Shutdown시키면 spfile이 재생성 됨

D-20 (하)	DBMS > 3. 옵션 관리
	인가되지 않은 Object Owner의 제한
개요	
점검 내용	Object Owner가 인가된 계정에게만 존재하는지 점검
점검 목적	Object Owner가 비인가자에게 존재하고 있는 경우 중요 데이터에 대한 무단 접근이 가능하여 데이터의 일관성 및 무결성을 해치는 위험이 발생할 수 있으므로 비인가 된 계정의 Object Owner를 제한하여 내부 및 외부의 보안 위협을 최소화하기 위함
보안 위협	Object Owner는 SYS, SYSTEM과 같은 데이터베이스 관리자 계정과 응용 프로그램의 관리자 계정에만 존재하여야 하며, 일반 계정이 존재할 경우 공격자가 이를 이용하여 Object의 수정, 삭제가 가능하므로 중요 정보의 유출 및 변경의 위험이 존재함
참고	※ Object(객체) : ALTER, DELETE, EXECUTE, INDEX, INSERT, SELECT 등을 말함
점검 대상 및 판단 기준	
대상	Oracle DB, Altibase, Tiber, PostgreSQL 등
판단 기준	양호 : Object Owner가 SYS, SYSTEM, 관리자 계정 등으로 제한된 경우
	취약 : Object Owner가 일반 사용자에게도 존재하는 경우
조치 방법	Object Owner를 SYS, SYSTEM, 관리자 계정으로 제한 설정
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) 설정 확인(SQL*Plus)

```
SQL> SELECT DISTINCT owner FROM dba_objects WHERE owner NOT IN ('SYS', 'SYSTEM', 'MD
SYS', 'CTXSYS', 'ORDSYS', 'ORDPLUGINS', 'AURORA$JIS$UTILITY$', 'HR', 'ODM', 'ODM_MTR', '
OE', 'APDBA', 'OLAPSYS', 'OSE$HTTP$ADMIN', 'OUTLN', 'LBACSYS', 'MTSYS', 'PM', 'PUBLIC', '
QS', 'QS_ADM', 'QS_CB', 'QS_CBADM', 'DBSNMP', 'QS_CS', 'QS_ES', 'QS_OS', 'QS_WS', 'RMAN', 'S
H', 'WKSYS', 'WMSYS', 'XDB') AND owner NOT IN (SELECT grantee FROM ba_role_privs WHERE
granted_role = 'DBA');
```

Step 2) 권한 취소

```
SQL> REVOKE <권한> ON <Object> FROM user;
```

● Altibase

Step 1) 사용자에게 부여된 Object 권한 정보 확인

```
SELECT * FROM system_.sys_grant_object_;
```

```
SELECT * FROM system_.sys_privileges_;
```

Step 2) 부여된 권한 ID를 확인하여 불필요한 권한 회수

```
REVOKE <권한> ON <Object> FROM [소유자];
```

Priv ID	Object Privilege	Table	Sequence	PSM/ External Procedure	View	Directory	External Library
101	ALTER	○	○				
102	DELETE	○					
103	EXECUTE			○			○
104	INDEX	○					
105	INSERT	○					
106	REFERENCES	○					
107	SELECT	○	○		○		
108	UPDATE	○					
109	READ					○	
110	WRITE					○	

모든 사용자는 자동으로 데이터 테이블에 대한 SELECT 권한을 가진다.

[ALTIBASE HDB 지원 Object 접근 권한]

● Tiberio

Step 1) 데이터베이스 내 모든 스키마 Object 특권의 정보를 조회하여 인가받지 않은 Object 권한 소유자가 있는지 확인

```
SELECT * FROM dba_tbl_privs;
```

Step 2) 잘못된 Object 권한 소유자 발견 시 권한 회수

● PostgreSQL

Step 1) Object 권한 정보 확인

```
postgres=# SELECT DISTINCT reowner FROM pg_class WHERE reowner NOT IN (SELECT  
usesysid FROM pg_user WHERE usesuper = TRUE);
```

Step 2) 잘못된 Object 권한 소유자 발견 시 권한 회수

D-21 (중)	DBMS > 3. 옵션 관리
	인가되지 않은 GRANT OPTION 사용 제한
개요	
점검 내용	일반 사용자에게 GRANT OPTION이 ROLE에 의하여 부여되어 있는지 점검
점검 목적	GRANT OPTION을 ROLE에 의해 설정하여 권한의 남용을 방지하고, 안정성을 확보하기 위함
보안 위협	일반 사용자에게 GRANT OPTION이 부여된 경우, 일반 사용자가 Object 소유자인 것과 같이 다른 일반 사용자에게 권한을 부여할 수 있어 권한의 무분별한 확산으로 인한 중요 정보의 유출 등의 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, MySQL, Altibase, Tiberio 등
판단 기준	양호 : WITH_GRANT_OPTION이 ROLE에 의하여 설정된 경우
	취약 : WITH_GRANT_OPTION이 ROLE에 의하여 설정되지 않은 경우
조치 방법	WITH_GRANT_OPTION이 ROLE에 의하여 설정되도록 변경
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB, Tiberio

Step 1) 설정 확인(SQL*Plus)

```
SQL> SELECT grantee || ':' || owner || ':' || table_name FROM dba_tab_privs WHERE grantable = 'YES'
and owner NOT IN ('SYS', 'MDSYS', 'ORDPLUGINS', 'ORDSYS', 'SYSTEM', 'WMSYS', 'SDB',
'LBACSYS') AND grantee NOT IN (SELECT grantee FROM dba_role_privs WHERE granted_role =
'DBA') ORDER BY grantee; (계정이 나오는 경우 취약)
```

Step 2) 권한 회수, 재부여(SQL*Plus)

```
SQL> REVOKE role FROM user;
```

●

● MySQL

Step 1) 설정 확인

SELECT user, grant_priv FROM mysql.user; (계정이 나오는 경우 취약)

Step 2) 권한 회수

REVOKE <권한> ON <대상> FROM [계정명];

● Altibase

Step 1) 사용자 계정을 조회하여 일반 사용자에게 with grant option이 부여(1)되어 있는 경우 취약

SELECT * FROM system_.sys_users_;

SELECT * FROM system_.sys_grant_object_;

SELECT * FROM system_.sys_privileges_;

Step 2) 권한 회수

REVOKE <권한> ON <Object> FROM [계정명];

D-22 (하)	DBMS > 3. 옵션 관리
	데이터베이스의 자원 제한 기능을 TRUE로 설정
개요	
점검 내용	RESOURCE_LIMIT 값이 TRUE로 설정되어 있는지 점검
점검 목적	RESOURCE_LIMIT 값을 TRUE로 설정하여 자원의 과도한 사용을 방지하여 데이터베이스의 안정성을 보장하고, 효율적인 자원 관리를 수행하기 위함
보안 위협	자원 제한 기능을 TRUE로 설정하지 않을 경우, 특정 사용자가 과도하게 많은 자원을 소비할 수 있으며 이로 인해 시스템에 과부하가 발생할 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB
판단 기준	양호 : RESOURCE_LIMIT 설정이 TRUE로 되어있는 경우
	취약 : RESOURCE_LIMIT 설정이 FALSE로 되어있는 경우
조치 방법	RESOURCE_LIMIT 설정을 TRUE로 설정 변경
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) init.ora 설정 파일에 RESOURCE_LIMIT = TRUE 라인 추가

(Oracle_HomeDirectory/admin/pfile/init.ora)

#vi /Oracle_HomeDirectory/admin/pfile/init.ora

Step 2) SQL*Plus에서 명령어로 설정 추가 및 확인

SQL> ALTER SYSTEM SET RESOURCE_LIMIT = TRUE;

SQL> SHOW PARAMETER RESOURCE_LIMIT;

D-23 (상)	DBMS > 3. 옵션 관리
	xp_cmdshell 사용 제한
개요	
점검 내용	xp_cmdshell의 사용 여부 점검
점검 목적	불필요하게 활성화되어 있는 xp_cmdshell를 제한하여 공격자의 무단 접근 및 악성코드의 실행 위험을 감소시키기 위함
보안 위험	해킹 툴에서 자주 이용되고 있으며, 권한 상승이나 데이터 유출 등의 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	MSSQL
판단 기준	양호 : xp_cmdshell이 비활성화 되어 있거나, 활성화 되어 있으면 다음의 조건을 모두 만족하는 경우 1. public의 실행(Execute) 권한이 부여되어 있지 않은 경우 2. 서비스 계정(애플리케이션 연동)에 sysadmin 권한이 부여되어 있지 않은 경우
	취약 : xp_cmdshell이 활성화 되어 있고, 양호의 조건을 만족하지 않는 경우
조치 방법	xp_cmdshell 설정 값을 0 또는 False로 설정
조치 시 영향	xp_cmdshell을 사용하여 운영체제 명령을 실행하던 서비스 및 스크립트가 동작하지 않을 수 있음

점검 및 조치 사례

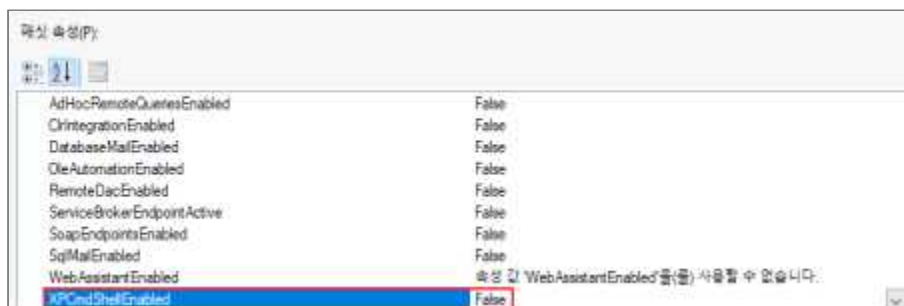
● MSSQL

[xp_cmdshell 사용이 불필요한 경우]

Step 1) SQL Server Management Studio > 개체 탐색기 > 컴퓨터 이름 우클릭 > 패싯 > 일반

Step 2) XPCmdShellEnabled 값 확인

2.1) Microsoft SQL Server Management Studio에서 확인



[개체 탐색기를 통한 프로시저 확인]

2.2) 쿼리문으로 확인

```
SELECT name, value FROM sys.configurations WHERE name = 'xp_cmdshell';
```

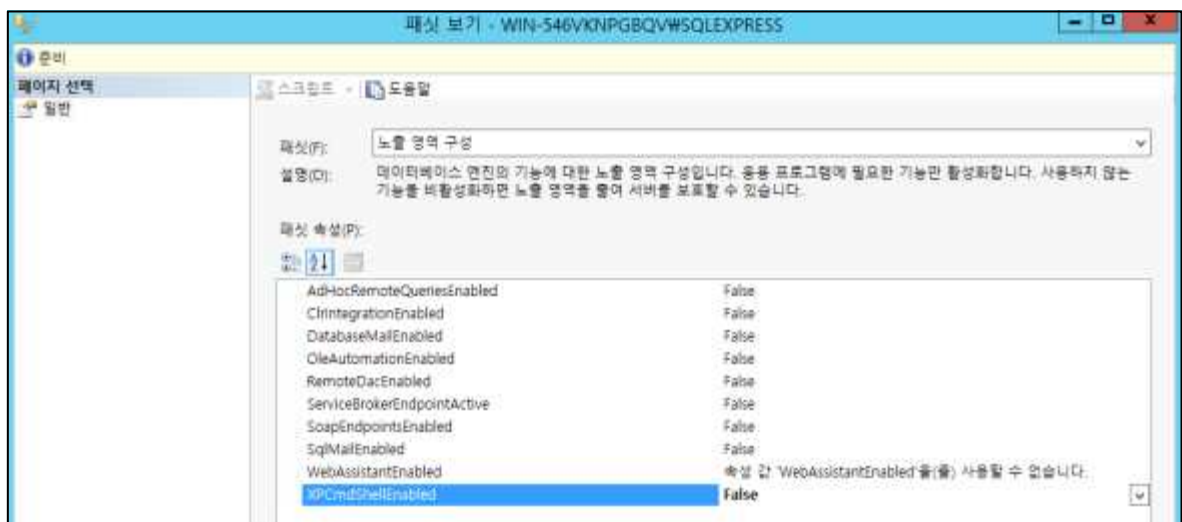
결과		메시지
	name	value
1	xp_cmdshell	1

※ value가 1이면 활성화, 0이면 비활성화 되어 있는 상태

Step 3) XPCmdShellEnabled 값을 false로 설정

3.1) Microsoft SQL Server Management Studio에서 설정

SQL Server Management Studio > 개체 탐색기 > 컴퓨터 이름 우클릭 > 패싯 > 일반



3.2) 쿼리문으로 설정

```
EXEC sp_configure 'show advanced options', 1; GO
```

```
RECONFIGURE; GO
```

```
EXEC sp_configure 'xp_cmdshell', 1; GO
```

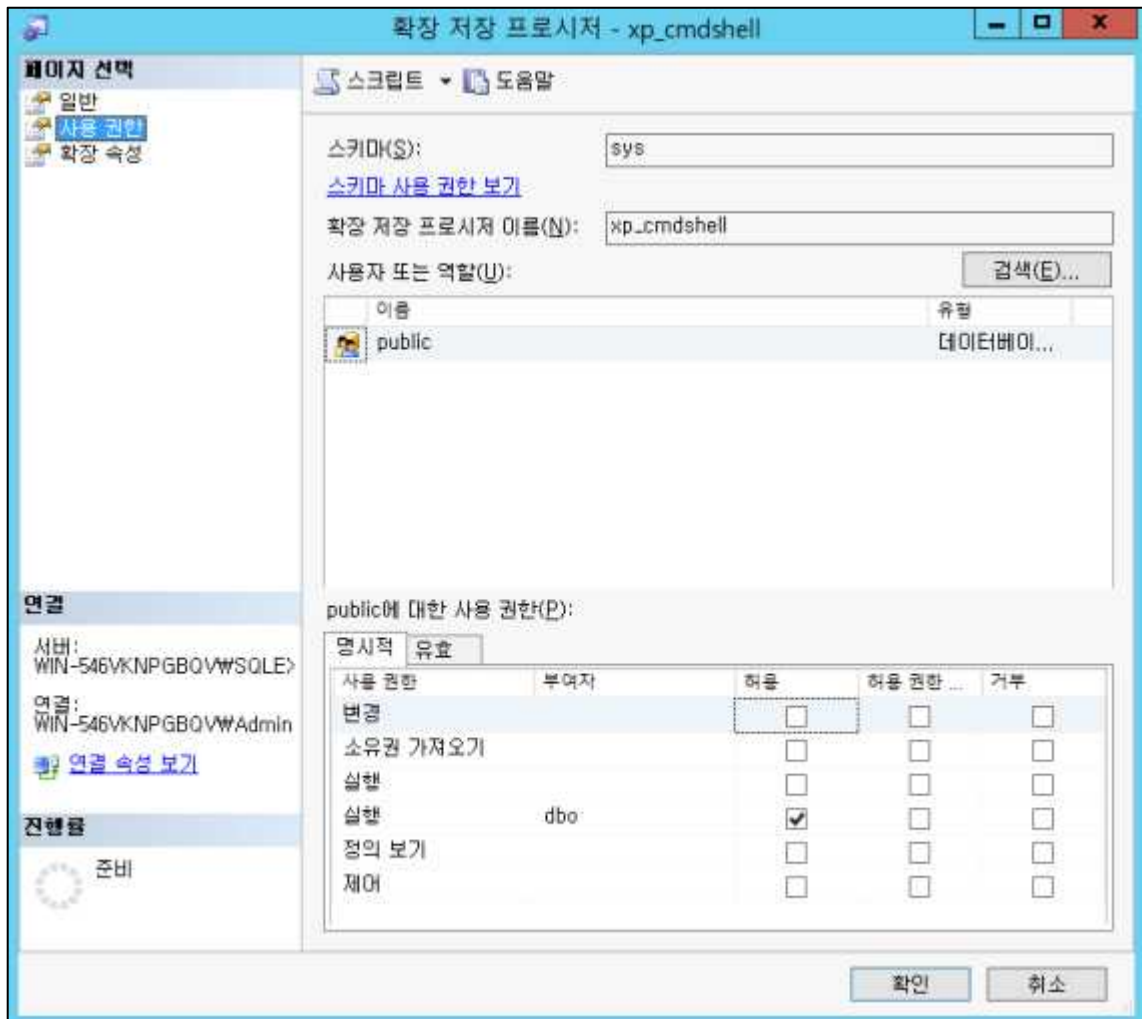
```
RECONFIGURE GO
```

[xp_cmdshell 사용이 필요한 경우]

Step 1) xp_cmdshell의 public 실행 권한 제거

1.1) Microsoft SQL Server Management Studio에서 제거

SQL Server Management Studio > 개체 탐색기 > [컴퓨터 이름] > 데이터베이스 > 시스템 데이터베이스 > master > 프로그래밍 기능 > 확장 저장 프로시저 > 시스템 확장 저장 프로시저 > sys.xp_cmdshell > 마우스 우클릭 > 속성 > 사용권한에서 public에 대한 사용권한에 '실행' 권한 제거



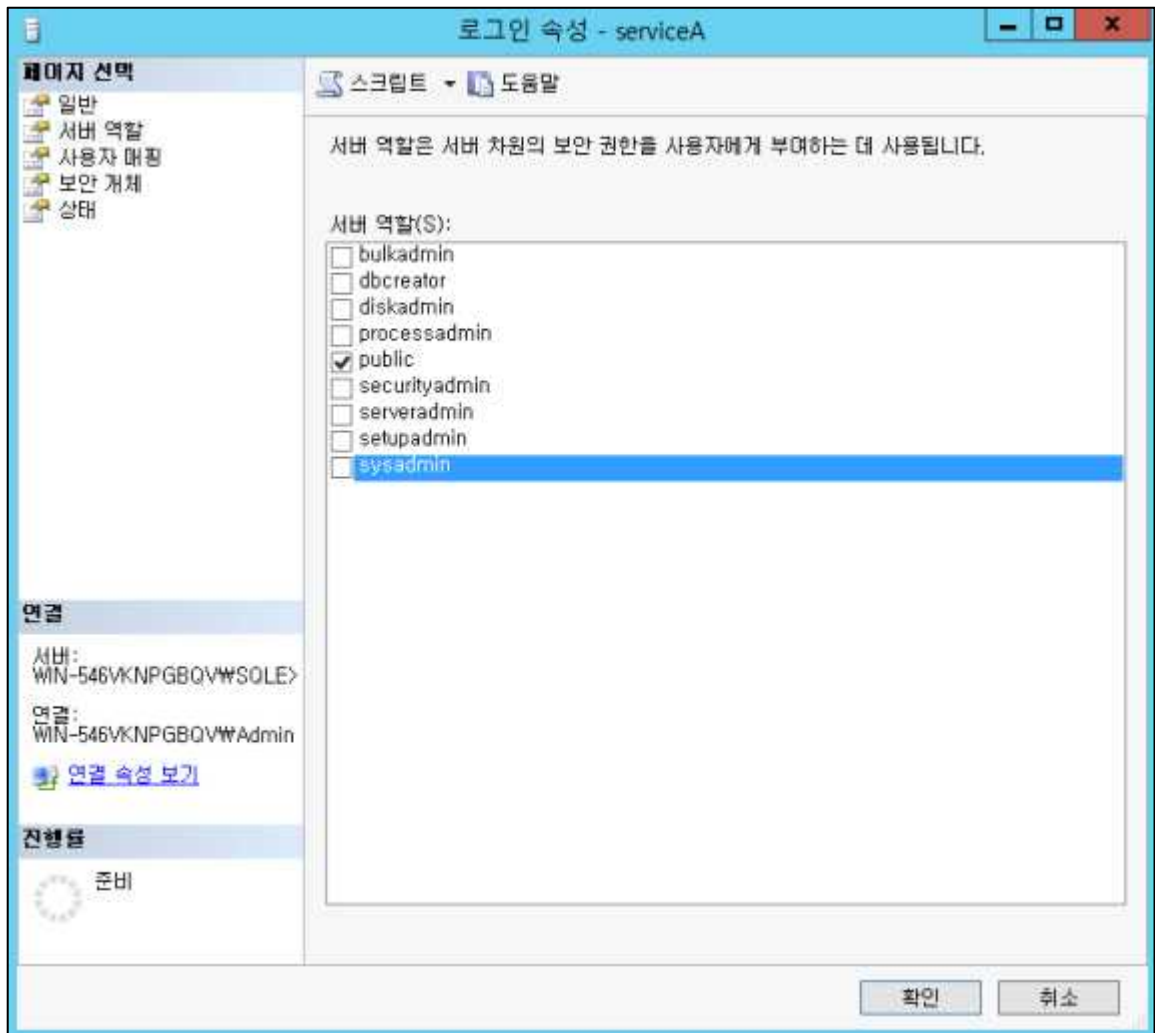
1.2) 쿼리문으로 public에 대한 실행 권한 제거

REVOKE EXECUTE ON master.dbo.xp_cmdshell TO public

Step 1) 서비스 계정(애플리케이션 연동 등)의 sysadmin 권한 제거

2.1) Microsoft SQL Server Management Studio에서 제거

SQL Server Management Studio > 개체 탐색기 > [컴퓨터 이름] > 보안 > 로그인 > [각 계정 선택] > 마우스 우클릭 > 속성 > 서버 역할에서 sysadmin 권한 제거



2.2) 쿼리문으로 서비스 계정의 sysadmin 권한 제거

- sysadmin 권한이 부여된 계정 확인

EXEC sp_helpsrvrolemember 'sysadmin'

결과		메시지	
	ServerRole	MemberName	MemberSID
1	sysadmin	sa	0x01
2	sysadmin	WIN-546VKNPGBQVWAdministrator	0x010500000000000515000000A66AF02031B3A55A1B98986...
3	sysadmin	NT SERVICE\SQLWriter	0x010600000000000550000000732B9753646EF90356745CB...
4	sysadmin	NT SERVICE\Winmgmt	0x0106000000000005500000005A048DDFF9C7430AB450D4...
5	sysadmin	NT Service\MSSQL\$SQLEXPRESS	0x010600000000000550000000703344E71D40B7FFB884456...
6	sysadmin	serviceA	0xCA8DE10CFD399344B7848C0867E69592

- sysadmin 권한이 부여된 계정에 대해 권한 제거

EXEC master..sp_drop_srvrolemember @loginame = N'<계정명>', @rolename = N'sysadmin'

※

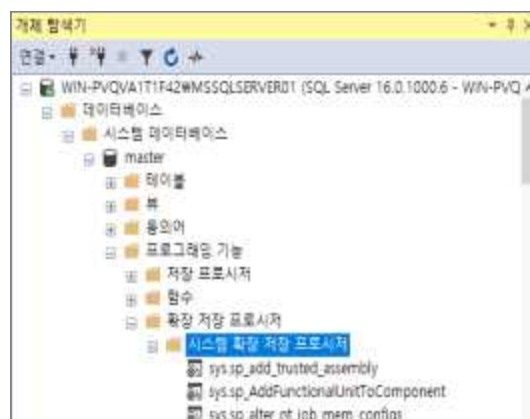
D-24 (상)	DBMS > 3. 옵션 관리
	Registry Procedure 권한 제한
개요	
점검 내용	Registry Procedure의 권한 설정 확인 및 점검
점검 목적	불필요한 Registry Procedure의 권한 설정을 확인하고 제한하여 시스템의 보안 및 안정성을 강화하기 위함
보안 위협	불필요한 레지스트리 접근 권한이 제한되지 않는 경우, 공격자가 시스템을 변경하거나 악성 소프트웨어를 설치하여 권한 상승, 데이터 유출, 시스템 장애를 발생시킬 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	MSSQL
판단 기준	양호 : 제한이 필요한 시스템 확장 저장 프로시저들이 DBA 외 guest/public에게 부여되지 않은 경우
	취약 : 제한이 필요한 시스템 확장 저장 프로시저들이 DBA 외 guest/public에게 부여된 경우
조치 방법	guest/public에게 부여된 시스템 확장 저장 프로시저 권한 제거
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● MSSQL

Step 1) SQL Server Management Studio > 개체 탐색기 > 데이터베이스

Step 2) 시스템 데이터베이스 > master > 프로그래밍 기능 > 확장 저장 프로시저 > 시스템 확장 저장 프로시저



[시스템 확장 저장 프로시저 확인]

Step 3) 각 시스템 확장 저장 프로시저 제한 > 마우스 우클릭 > 속성



[시스템 확장 저장 프로시저 속성 확인]

Step 4) 사용 권한 > public 실행 권한 제거(체크 해제)



[public 실행 권한 제거]

시스템 확장 저장 프로시저 제한		
sys.xp_readdmultistring	sys.xp_redeletekey	sys.xp_regdeletevalue
sys.xp_regenumvalues	sys.xp_regread	sys.xp_regremovemultistring
sys.xp_regwrite		

D-25 (상)	DBMS > 4. 패치 관리
	주기적 보안 패치 및 벤더 권고 사항 적용
개요	
점검 내용	안전한 버전의 데이터베이스를 사용하고 있는지 점검
점검 목적	안전한 버전의 데이터베이스를 사용하여 알려진 보안 취약점으로 인한 공격을 차단하기 위함
보안 위협	안전하지 않은 버전을 사용할 경우, 알려진 보안 취약점을 통해 시스템에 침투하거나 데이터의 탈취, 악성코드 감염 및 서비스 중단 등의 보안 사고를 초래할 위험이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, MySQL, Altibase, Tiberio, PostgreSQL, Cubrid 등
판단 기준	양호 : 보안 패치가 적용된 버전을 사용하는 경우
	취약 : 보안 패치가 적용되지 않는 버전을 사용하는 경우
조치 방법	보안 패치가 적용된 버전으로 업데이트
조치 시 영향	기존 시스템 운영 등에 사용되던 시스템 구성 요소와 호환성 문제가 발생할 수 있음

점검 및 조치 사례

● Oracle DB

Step 1) 시스템에서 제품 버전 현황 확인(SQL*Plus)

```
SQL> SELECT banner FROM v$version WHERE banner LIKE 'Oracle%';
```

Step 2) Oracle 최신 버전 확인

<http://www.oracle.com/technetwork/database/enterprise-edition/downloads/index.html>

● MSSQL

Step 1) 시스템에서 제품 버전 현황 확인

```
SELECT @@version
```

또는

```
SELECT SERVERPROPERTY('productversion') AS ProductVersion, SERVERPROPERTY('productlevel') AS ProductLevel, SERVERPROPERTY('edition') AS Edition;
```

Step 2) MSSQL 최신 버전 확인

<http://support.microsoft.com/kb/321185/en-uswnloads/index.html>

● MySQL

Step 1) 시스템에서 제품 버전 현황 확인

```
mysql> SELECT VERSION();
```

Step 2) MySQL 최신 버전 확인

버그 패치된 릴리즈 사이트 <http://downloads.mysql.com/archives.php>

● Altibase

Step 1) 시스템에서 제품 버전 현황 확인

```
SELECT PRODUCT_SIGNATURE FROM v$database;
```

Step 2) Altibase 최신 패치 노트 확인

<http://support.altibase.com/kr/patch-note>

Step 3) 패키지 인스톨러를 이용한 제품 패치

- Altibase HDB 는 제품 패치를 위한 설치 파일이 따로 존재하지 않으며, 인스톨러를 시작할 때 설치 형태를 풀(full) 패키지 또는 패치로 선택할 수 있음

- Altibase 고객지원서비스 포털 (<http://support.altibase.com/>)을 방문하여 본인의 운영체제에 적합한 인스톨러를 다운로드 받을 수 있음

● Tiberio

Step 1) 시스템에서 제품 버전 현황 확인

```
tbboot -v
```

Step 2) Tiberio 최신 패치 노트 확인

<http://technet.tmaxsoft.com/>

※ Tiberio 패치 정책 (2015.02)

매 분기 초 픽스셋 발표(년간 총 4회 배포 / fixset : hot fix 모음)

● PostgreSQL

Step 1) 시스템에서 제품 버전 현황 확인

```
SELECT VERSION();
```

Step 2) PostgreSQL 최신 버전 확인

<http://www.postgresql.org/support/security>

- Cubrid

Step 1) 시스템에서 제품 버전 현황 확인

`cubrid_rel`

Step 2) Cubrid 최신 버전 확인

패치된 릴리즈 확인 및 다운로드 https://cubrid.com/release_note/

D-26 (상)	DBMS > 4. 패치 관리
	데이터베이스의 접근, 변경, 삭제 등의 감사 기록이 기관의 감사 기록 정책에 적합하도록 설정
개요	
점검 내용	감사 기록 정책 설정이 기관 정책에 적합하게 설정되어 있는지 점검
점검 목적	데이터, 로그, 응용 프로그램에 대한 감사 기록 정책을 수립하고 적용하여 데이터베이스에 문제 발생 시 원활하게 대응하기 위함
보안 위협	감사 기록 정책이 설정되어 있지 않을 경우, 데이터베이스에 문제 발생 시 원인을 규명할 수 있는 자료가 존재하지 않아 이에 대한 대처 및 개선방안 수립이 어려워 장기적으로 심각한 보안 위협이 존재함
참고	-
점검 대상 및 판단 기준	
대상	Oracle DB, MSSQL, Altibase, Tiberio, PostgreSQL 등
판단 기준	양호 : DBMS의 감사 로그 저장 정책이 수립되어 있으며, 정책 설정이 적용된 경우
	취약 : DBMS에 대한 감사 로그 저장을 하지 않거나, 정책 설정이 적용되지 않은 경우
조치 방법	DBMS에 대한 감사 로그 저장 정책 수립, 적용
조치 시 영향	일반적인 경우 영향 없음

점검 및 조치 사례

● Oracle DB

Step 1) 데이터베이스 감사 기록 정책 및 백업 정책 수립

Step 2) DBMS에 대한 기본적인 감사를 설정함

아래와 같은 명령어를 통해 로그인 실패, 권한, Object 등에 대한 감사 설정

```
SQL> connect sys as sysdba
```

```
Enter password: ****
```

```
Connected.
```

```
SQL> ALTER SYSTEM SET AUDIT_TRAIL=DB SCOPE=SPFILE;
```

```
System altered.
```

```
SQL> shutdown immediate
```

```
Database closed.
```

```
Database dismounted.
```

```
ORACLE instance shut down.
```

SQL> startup

ORACLE instance started.

SQL> AUDIT SESSION WHENEVER NOT SUCCESSFUL;

Audit succeeded.

● MSSQL

데이터베이스 감사 기록 정책 및 백업 정책 수립

● MSSQL 2000

DB 접근에 대한 보안 감사를 할 수 있도록 보안 감사 설정

[SQL SERVER] > [등록정보] > [보안] > [감사수준] > '모두' 선택

● MSSQL 2005, 2008, 2012, 2016, 2019, 2022

[SQL SERVER] > [마우스 우클릭] > [속성] > [보안] > [로그인 감사] 옵션 > '실패한 로그인과 성공한 로그인 모두' 선택



[로그인 감사 설정]

● Altibase

Step 1) AUDIT 구문으로 감사 정책을 설정

Step 2) 정책 설정 후 감사 조건 적용

ALTER SYSTEM STOP AUDIT;

ALTER SYSTEM START AUDIT;

ALTER SYSTEM RELOAD AUDIT;

※ Altibase HDB 서버 내에서 실행되고 있는 특정 구문 또는 모든 구문을 실시간으로 추적하고, 로그를 남기는 것을 감사(Audit)라고 하며, SYS 사용자만이 이 구문을 사용해서 감사 조건을 설정할 수 있음

● Tiberio

Step 1) 감사 기능은 감사 대상에 따라 두 종류로 구분됨

1. 스키마 Object에 대한 감사

지정된 스키마 Object에 수행되는 모든 동작을 기록할 수 있음

2. 시스템 특권에 대한 감사

지정된 시스템 특권을 사용하는 모든 동작을 기록할 수 있음

※ 감사를 설정하거나 해제하려면 다음 명령을 사용함

- audit (감사 설정)

- noaudit (감사 해제)

[감사 설정]

Step 1) 스키마 Object에 대한 감사

다른 사용자가 소유한 스키마의 Object 또는 디렉터리 Object를 감사하기 위해서는 AUDIT ANY 시스템 특권을 부여받아야 함

예시) `AUDIT DELETE ON t BY SESSION WHENEVER SUCCESSFUL;`

→ 테이블에 수행되는 모든 DELETE 문이 성공하는 경우에만 감사 기록을 남김

Step 2) 시스템 특권에 대한 감사

시스템 특권을 감사하기 위해서는 AUDIT SYSTEM 시스템 특권을 부여받아야 함

예시) `AUDIT CREATE table BY Tiberio;`

→ Tiberio라는 사용자가 테이블을 생성하려고 할 때 그것이 성공하든 실패하든 관계없이 감사 기록을 남김

[감사 해제]

Step 1) 스키마 Object에 대한 감사 해제

다른 사용자가 소유한 스키마의 Object 또는 디렉터리 Object의 감사를 해제하기 위해서는 AUDIT ANY 시스템 특권을 부여받아야 함

예시) `NOAUDIT DELETE ON t BY SESSION WHENEVER SUCCESSFUL;`

→ 테이블에 수행되는 모든 DELETE문에 대해 더 이상 감사 기록을 남기지 않음

Step 2) 시스템 특권에 대한 감사 해제

시스템 특권의 감사를 해제하기 위해서는 AUDIT SYSTEM 시스템 특권을 부여받아야 함

예시) `NOAUDIT CREATE table BY Tiberio;`

→ Tiberio라는 사용자가 테이블을 생성할 때 더 이상 감사 기록을 남기지 않음

※ SYS 사용자 감사 설정 방법

<\$TB_SID.tip> 파일을 아래 내용처럼 입력 또는 수정

예시) AUDIT_SYS_OPERATIONS=Y

AUDIT_FILE_DEST=/home/Tibero/audit/audit_trail.log

AUDIT_FILE_SIZE=10M

SYS사용자의 명령을 감사하도록 설정하면 수행한 모든 동작이 OS 파일에 기록되며 보안상의 이유로 데이터 베이스에는 기록되지 않음

● PostgreSQL

Step 1) Log 감사 설정 여부 확인

```
postgres=# SHOW logging_collector;
```

```
logging_collector
```

```
-----
```

```
on (1 row)
```

Step 2) postgresql.conf 파일 내 logging_collector을 on으로설정

```
logging_collector = on
```